



ADLINK
TECHNOLOGY INC.

WD-DASK

Software Development Kit for
PCI/PCIe/PXI Digitizer Data Acquisition Cards

Function Library Reference

Revision: 4.50
Revision Date: Mar. 25, 2015
Part No: 50-11225-2090

Advance Technologies; Automate the World.

Revision History

Revision	Release Date	Change(s)
2.00	05/17/2010	API for PCI/PXI 98x6 added
3.00	09/16/2011	API for PCIe-9842 API added, 64-bit library added, support added for wd- dask.h for x86 and wd-dask64.h for x64, and titles and nomenclature normalized.
3.10	09/21/2012	API for PCIe-9842 and PXIe-9848 added
3.30	03/27/2013	API for PCIe/PXIe-9852 added
3.50	07/19/2013	Functions added, reorganized
4.00	01/26/2014	Functions added for PXIe-9852 release
4.50	03/25/2015	Functions added for PCIe-9814 release

Preface

Copyright 2015 ADLINK Technology, Inc.

This document contains proprietary information protected by copyright. All rights are reserved. No part of this manual may be reproduced by any mechanical, electronic, or other means in any form without prior written permission of the manufacturer.

Disclaimer

The information in this document is subject to change without prior notice in order to improve reliability, design, and function and does not represent a commitment on the part of the manufacturer.

In no event will the manufacturer be liable for direct, indirect, special, incidental, or consequential damages arising out of the use or inability to use the product or documentation, even if advised of the possibility of such damages.

Environmental Responsibility

ADLINK is committed to fulfill its social responsibility to global environmental preservation through compliance with the European Union's Restriction of Hazardous Substances (RoHS) directive and Waste Electrical and Electronic Equipment (WEEE) directive. Environmental protection is a top priority for ADLINK. We have enforced measures to ensure that our products, manufacturing processes, components, and raw materials have as little impact on the environment as possible. When products are at their end of life, our customers are encouraged to dispose of them in accordance with the product disposal and/or recovery programs prescribed by their nation or company.

Trademarks

Product names mentioned herein are used for identification purposes only and may be trademarks and/or registered trademarks of their respective companies.

Conventions

Take note of the following conventions used throughout this manual to make sure that users perform certain tasks and instructions properly.



NOTE:

Additional information, aids, and tips that help users perform tasks.



CAUTION:

Information to prevent **minor** physical injury, component damage, data loss, and/or program corruption when trying to complete a task.



WARNING:

Information to prevent **serious** physical injury, component damage, data loss, and/or program corruption when trying to complete a specific task.

Table of Contents

Revision History ii.....	ii
Preface	iii
List of Tables.....	ix
1 Introduction	1
1.1 Application Building Fundamentals in Windows	1
Using Microsoft® Visual C®/C++®	1
Using Microsoft® Visual Basic®	2
Creating Microsoft® .NET Applications	4
1.2 Application Building Fundamentals in Linux	6
1.3 Function Classes	6
2 Function Library	9
2.1 Data Types	9
2.2 List of Functions.....	11
2.3 Function Library	12
2.3.1 Analog Input	12
WD_AI_Array_Scale	12
WD_AI_AsyncCheck	14
WD_AI_AsyncClear	15
WD_AI_AsyncDbiBufferHalfReady	16
WD_AI_AsyncDbiBufferHandled	17
WD_AI_AsyncDbiBufferMode	18
WD_AI_AsyncDbiBufferOverrun	19
WD_AI_AsyncDbiBufferToFile	20
WD_AI_AsyncReStartNextReady	21
WD_AI_AsyncReTrigNextReady	22
WD_AI_CH_ChangParam	23
WD_AI_CH_Config	25

WD_AI_Config	26
WD_AI_ContBufferReset	29
WD_AI_ContBufferSetup	30
WD_AI_ContBufferSetup32	31
WD_AI_ContReadChannel	32
WD_AI_ContReadChannelToFile	35
WD_AI_ContReadMultiChannels	39
WD_AI_ContReadMultiChannelsToFile	43
WD_AI_ContScanChannels	47
WD_AI_ContScanChannelsToFile	51
WD_AI_ContStatus	55
WD_AI_ContVScale	59
WD_AI_ContVScaleEx	60
WD_AI_ConvertCheck	62
WD_AI_DMA_Transfer	62
WD_AI_DMA_TransferBySize	63
WD_AI_EventCallBack	66
WD_AI_InitialMemoryAllocated	67
WD_AI_Set_Mode	68
WD_AI_SetTimeout	69
WD_AI_Trig_Config	70
WD_AI_VoltScale	74
WD_AI_VoltScale32	75
WD_OutTrig_Config	76
2.3.2 Digital Input/ Output.....	77
WD_DI_ReadLine	77
WD_DI_ReadPort	78
WD_DIO_LineConfig	79
WD_DIO_LinesConfig	80
WD_DIO_PortConfig	81
WD_DO_ReadLine	82
WD_DO_ReadPort	83

2.3.3	Configuration	84
	WD_Register_Card	84
	WD_Register_Card_By_PXISlot_GA	87
	WD_Release_Card	88
	WD_Buffer_Alloc	89
	WD_Buffer_Free	89
2.3.4	Calibration	90
	WD_AD_Auto_Calibration_ALL	90
	WD_EEPROM_CAL_Constant_Update	91
	WD_Get_Default_Load_Area	92
	WD_Load_CAL_Data	92
	WD_Set_Default_Load_Area	93
2.3.5	SSI	94
	WD_Route_Signal	94
	WD_Signal_DisConn	96
	WD_SSI_SourceClear	99
	WD_SoftTriggerGen	99
	WD_SSI_SourceConn	100
	WD_SSI_SourceDisConn	101
2.3.6	Other	102
	WD_Device_Scan	102
	WD_Get_SDRAMSize	103
	WD_GetSerialNumber	104
	WD_GetPXIGeographAddr	105
	WD_GetFPGAVersion	106
	WD_MeasureStats	106
	WD_SoftTriggerGen	107
A	Appendix: Codes and Formats	109
A.1	Status Codes	109
A.2	AI Range Codes	114
A.3	AI Data Format	116

A.4	Data File Format	117
	Header	117
	Channel Range	118
	Data Block	119
	Important Safety Instructions.....	121
	Getting Service	123

List of Tables

Table 2-1: WD_AI_ContReadChannel ReadScan	
Module Restrictions	34
Table 2-2: WD_AI_ContReadChannelToFile ReadScan	
Module Restrictions	38
Table 2-3: WD_AI_ContReadMultiChannels ReadScan	
Module Restrictions	42
Table 2-4: WD_AI_ContReadMultiChannelsToFile ReadScan	
Module Restrictions	45
Table 2-5: WD_AI_ContScanChannels ReadScan	
Module Restrictions	49
Table 2-6: WD_AI_ContScanChannelsToFile ReadScan	
Module Restrictions	53
Table A-1: Status Codes	113
Table A-2: Analog Digitizer Card Input Ranges	115
Table A-3: Valid Card Values	115
Table A-4: Header Data Structure	118
Table A-5: Channel Range	119

This page intentionally left blank.

1 Introduction

WD-DASK is a driver application for ADLINK digitizer data acquisition cards, providing high performance data acquisition for developing custom applications in Windows® and Linux environments.

WD-DASK leverages the power and flexibility of Windows® and Linux for challenging data acquisition applications. These include running multiple applications and using extended memory. In addition, implementing WD-DASK under Visual Basic environment makes it easy to create custom user interfaces and graphics.

1.1 Application Building Fundamentals in Windows

The following sections provide fundamental instructions when using WD-DASK to build application in Windows® operating environments.

Using Microsoft® Visual C®/C++®

Follow these steps to create a data acquisition application using WD-DASK and Microsoft Visual C/C++.

1. Launch the Microsoft Visual C/C++ application.
2. Open a new or existing project that you want to apply the WD-DASK.
3. Include the **wd-dask.h** for x86 or **wd-dask64.h** for x64 header file in the C/C++ source files that call WD-DASK functions. **wd-dask.h/wd-dask64.h** contains all the function declarations and constants that can be used to develop data acquisition applications. Incorporate the following statement in the code to include the header file.



NOTE:

- For X86: `#include "wd-dask.h"`
- For X64: `#include "wd-dask64.h"`

-
4. After setting the appropriate compile and link options, build the application by selecting the Build command

from Build menu. Remember to link WD-DASK's import library, wd-dask.lib for X86 and wd-dask64.lib for X64.

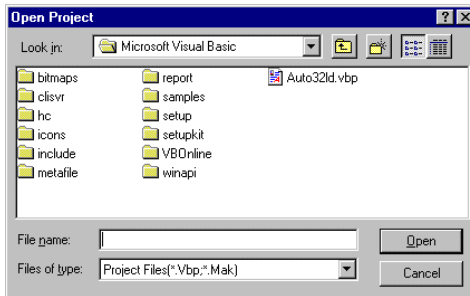
Using Microsoft® Visual Basic®

Follow the steps in the succeeding sections to create a data acquisition application using WD-DASK and Visual Basic.

Open a project

Do one of the following to open a new or existing project:

1. Open a new project by selecting the New Project command from the File menu. To open an existing project, select the Open Project command from the File menu to display the Open Project dialog box.



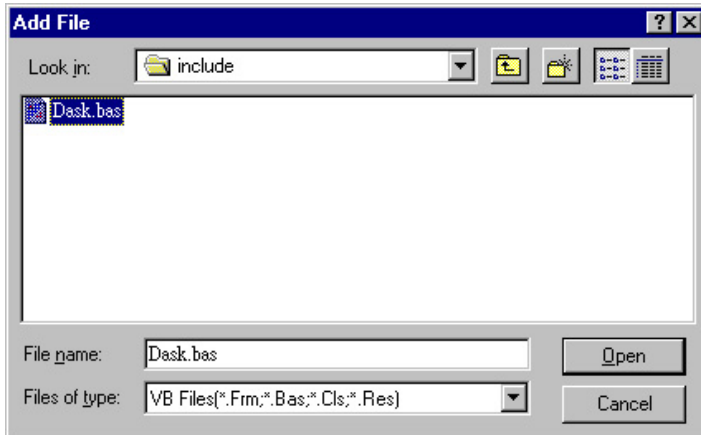
2. Locate the existing project, then double-click on the project file name to load.

Add the file

You must add the file wd-dask.bas to the project, if the file is not yet included. This file contains all the procedure declarations and constants that can be used to develop the data acquisition application. To add the file:

1. Select Add File from the File menu. The Add File window

appears, displaying a list of files in the current directory.



2. Double-click on the wd-dask.bas file. If the file is not on the list, make sure the list is displaying files from the correct directory. By default, the wd-dask.bas file is installed at C:\ADLINK\WD-DASK\INCLUDE.

Design the interface

To design the interface for the application, place all the interface elements such as command buttons, list boxes, and text boxes on the Visual Basic form. These standard controls are available from the Visual Basic Toolbox.

To place a control on the form, select the desired control from the Toolbox, then draw it on the form. You may also double-click on the control icon from the Toolbox to place it on the form.

Set the interface controls

To view the property list, click the desired control, then choose the Properties command from the View menu, or press F4. You may also click on the Properties button  from the toolbar.

Write the event code

The event code defines the required action to be performed when an event occurs. To write the event code, double-click on the control or form to view the code module, then add the event code. You

can also call the functions declared in the WD-DASK.BAS file to perform data acquisition operations.

Run the application

Do one of the following to run the application:

- ▶ Choose **Start** from the **Run** menu
- ▶ Click the Start icon  from the toolbar
- ▶ Press <F5>

Distribute the application

After completing the project, save the application as an executable (.EXE) file using the **Make EXE File** command from the File menu. The application, after being transformed into an executable file, is now ready for distribution.

You must include the WD-DASK's DLL and driver files when the application is distributed. Refer to WD-DASK User's Guide for details.

Creating Microsoft ® .NET Applications

The Microsoft .NET CLR allocates all the resources from the managed heap. An object is created in memory but there is no guarantee that the object will remain in the same location where it was created in. If a memory space is allocated to perform DAQ/DMA operation but the memory is moved by the GC, an unexpected memory exception error occurs.

To prevent the memory for DMA from being moved to other location, two methods of memory allocation can be utilized, as follows:

- ▶ Allocate memory from the unmanaged memory of the process:

Namespace: System.Runtime.InteropServices

`Marshal.AllocHGlobal/Marshal.FreeHGlobal`

- ▷ Use `Marshal.AllocHGlobal` to allocate memory and get a pointer to the buffer
- ▷ Pass the pointer to the wd-dask API which needs the buffer address
- ▷ Use `Marshal.FreeHGlobal` to free the memory.
- Using `GCHandle` class to pin the the buffer down in memory

Namespace: `System.Runtime.InteropServices`

`GCHandle.Alloc/GCHandle.Free`

- ▷ Use new operation to allocate an array.
- ▷ `GCHandle.Alloc` to get a pinned pointer to an array.
- ▷ Pass `handle.AddrOfPinnedObject()` to the wd-dask API which needs the buffer address.
- ▷ Use `GCHandle.Free` to releases a `GCHandle`.

Creating Microsoft® Visual C# Applications

Follow these steps to create a C# application using WD-DASK and Microsoft Visual Studio 2005 or later.

1. Launch Microsoft Visual C#.
2. Open a new or existing project that you want to apply the WD-DASK.
3. Add the WD-DASK C# class definition file `wd-dask.cs` (X86) or `wd_dask64.cs` (X64) to the project.
4. Build the application by selecting the Build command from Build menu.

Creating Microsoft® VB.Net Applications

Follow these steps to create a VB.Net application using WD-DASK and Microsoft Visual Studio 2005 or later.

1. Launch Microsoft VB.Net.
2. Open a new or existing project that you want to apply the WD-DASK.
3. Add the wd-dask vb.Net class definition file wd-dask.vb (X86) or wd-dask64.vb (X64) to the project.
4. Build the application by selecting the Build command from Build menu.

1.2 Application Building Fundamentals in Linux

The following sections provide fundamental instructions when using WD-DASK to build application in Linux. To create a data acquisition application using WD-DASK/X and GNU C/C++, follow these steps:

Edit the source files

Include the header file wd-dask.h in the C/C++ source files that call WD-DASK/X functions. The wd-dask.h has all the function declarations and constants that you can use to develop your data acquisition application. Add this statement in your code to include the header file.

```
#include "wd-dask.h"
```

Build your application

Using the appropriate C/C++ compiler (gcc or cc) to compile the program. You should add **-lwd_dask** option to link libwd_dask.so library. For multi-threaded applications, the **-lpthread** string is required. For example:

```
gcc -o testai testai.c -lwd_dask
```

1.3 Function Classes

This section describes the classes of functions that the WD-DASK supports.



NOTE:

Please note: all settings shown 4321, and ON=0, OFF=1

All WD-DASK functions are grouped into different classes:

- ▶ General Configuration Function Group
- ▶ Analog Input Function Group
 - ▷ Analog Input Configuration functions
 - ▷ One-Shot Analog Input functions
 - ▷ Continuous Analog Input functions
 - ▷ Asynchronous Analog Input Monitoring functions
- ▶ SSI Function Group
- ▶ Calibration Function Group

This page intentionally left blank.

2 Function Library

This chapter details the WD-DASK function library for development of application programs within Visual C++ and Visual Basic, including data types and function definitions.

2.1 Data Types

The WD-DASK library uses these data types in wd-dask.h/wd-dask64.h. It is recommended that you use these data types in your application programs. The table shows the data type names, ranges, and corresponding data types in C/C++, Visual Basic, and Delphi for your reference.

Type		Range	Type		
			C/C++ (for 32-bit compiler)	Visual Basic	Pascal (Delphi)
U8	8-bit ASCII character	0 to 255	unsigned char	Byte	Byte
I16	16-bit signed integer	-32768 to 32767	short	Integer	SmallInt
U16	16-bit unsigned integer	0 to 65535	unsigned short	Not supported by BASIC, use the signed integer (I16) instead	Word
I32	32-bit signed integer	-2147483648 to 2147483647	long	Long	LongInt
U32	32-bit unsigned integer	0 to 4294967295	unsigned long	Not supported by BASIC, use the signed long integer (I32) instead	Cardinal
F32	32-bit single-precision floating-point	3.402823E38 to 3.402823E38	float	Single	Single

Type		Range	Type		
			C/C++ (for 32-bit compiler)	Visual Basic	Pascal (Delphi)
F64	64-bit double-precision floating-point	1.797683134862315E308 to 1.797683134862315E309	Double	Double	Double



NOTE:

Data types in WD-DASK.BAS and WD-DASK.PAS are not defined, and are listed only for reference.

2.2 List of Functions

Function Group	
Analog Input (Section 2.3.1 Analog Input)	WD_AI_Array_Scale
	WD_AI_AsyncCheck
	WD_AI_AsyncClear
	WD_AI_AsyncDblBufferHalfReady
	WD_AI_AsyncDblBufferHandled
	WD_AI_AsyncDblBufferMode
	WD_AI_AsyncDblBufferOverrun
	WD_AI_AsyncDblBufferToFile
	WD_AI_AsyncReStartNextReady
	WD_AI_AsyncReTrigNextReady
	WD_AI_CH_ChangParam
	WD_AI_CH_Config
	WD_AI_Config
	WD_AI_ContBufferReset
	WD_AI_ContBufferSetup
	WD_AI_ContBufferSetup32
	WD_AI_ContReadChannel
	WD_AI_ContReadChannelToFile
	WD_AI_ContReadMultiChannels
	WD_AI_ContReadMultiChannelsToFile
	WD_AI_ContScanChannels
	WD_AI_ContScanChannelsToFile
	WD_AI_ContStatus
	WD_AI_ContVScale
	WD_AI_ContVScaleEx
	WD_AI_ConvertCheck
	WD_AI_DMA_Transfer
	WD_AI_DMA_TransferBySize
	WD_AI_EventCallBack
	WD_AI_InitialMemoryAllocated
	WD_AI_Set_Mode
	WD_AI_SetTimeout
	WD_AI_Trig_Config
	WD_AI_VoltScale
	WD_AI_VoltScale32
	WD_OutTrig_Config

Function Group	
Digital Input/ Output (Section 2.3.2 Digital Input/ Output)	WD_DI_ReadLine WD_DI_ReadPort WD_DIO_LineConfig WD_DIO_LinesConfig WD_DIO_PortConfig WD_DO_ReadLine WD_DO_ReadPort
Configuration (Section 2.3.3 Configuration)	WD_Register_Card WD_Register_Card_By_PXISlot_GA WD_Release_Card WD_Buffer_Alloc WD_Buffer_Free
Calibration (Section 2.3.4 Calibration)	WD_AD_Auto_Calibration_ALL WD_Get_Default_Load_Area WD_EEPROM_CAL_Constant_Update WD_Load_CAL_Data WD_Set_Default_Load_Area
SSI (Section 2.3.5 SSI)	WD_Route_Signal WD_Signal_DisConn WD_SoftTriggerGen WD_SSI_SourceClear WD_SSI_SourceConn WD_SSI_SourceDisConn
(Section 2.3.6 Other)Other	WD_Device_Scan WD_Get_SDRAMSize WD_GetSerialNumber WD_GetPXIGeographAddr WD_GetFPGAVersion WD_MeasureStats WD_SoftTriggerGen

2.3 Function Library

2.3.1 Analog Input

WD_AI_Array_Scale

Converts the values of an array of specified width of binary data from a continuous A/D conversion call to actual input

voltages. Can convert the data with the specified AI range of each channel.

Supported Cards

9820, 9816, 9826, 9846, 9842, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_Array_Scale (U16 CardNumber, U16
numRanges, U16 *rangeList, U16 width, void
*readingArray, F64 *voltageArray, I32 count)
```

Visual Basic

```
WD_AI_Array_Scale (ByVal CardNumber As Integer,
ByVal numRanges As Integer, rangeList As
Integer, ByVal width As Integer, readingArray
As Integer, voltageArray As Double, ByVal
count As Long) As Integer
```

Parameters

CardNumber:

ID of the card performing asynchronous operation.

numRanges:

Number of analog input ranges in the array rangeList

rangeList:

The analog input range array for the continuous specified channels. Please refer to Section A.2 AI Range Codes for valid range values.

width:

The data width of input data.

readingArray:

Acquired continuous analog input data array.

voltageArray:

Computed voltages array returned

count:

Amount of data to be converted

Return Code

NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
ErrorInvalidAdRange

WD_AI_AsyncCheck

Determines the current status of the asynchronous analog input operation.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_AsyncCheck (U16 CardNumber, BOOLEAN  
*Stopped, U32 *AccessCnt)
```

Visual Basic

```
WD_AI_AsyncCheck (ByVal CardNumber As Integer,  
Stopped As Byte, AccessCnt As Long) As Integer
```

Parameters

CardNumber:

ID of the card performing asynchronous operation.

Stopped:

Indicates whether the asynchronous analog input operation is completed. If Stopped = TRUE, the analog input operation has stopped after the number of A/D conversions indicated in the call that initiated the asynchronous analog input operation is complete or an error has occurred. If Stopped = FALSE, the operation is not yet completed. Constants TRUE and FALSE are defined in WD-DASK.h.

AccessCnt:

When trigger acquisition mode is not used, this parameter returns the number of A/D data that has been transferred at the time `WD_AI_AsyncCheck()` is called. When any trigger and double-buffered mode are enabled, `AccessCnt` returns to the next position after the last A/D data is stored in the circular buffer when `WD_AI_AsyncCheck()` is called.

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
```

WD_AI_AsyncClear

Stops the asynchronous analog input operation.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_AsyncClear (U16 CardNumber, U32
*StartPos, U32 *AccessCnt)
```

Visual Basic

```
WD_AI_AsyncClear (ByVal CardNumber As Integer,
StartPos As Long, AccessCnt As Long) As Integer
```

Parameters

CardNumber:

ID of the card performing asynchronous operation.

StartPos:

when trigger acquisition mode is not used, `StartPos` is zero. If "Pre-trigger or middle trigger" mode of AI is used, `StartPos` returns the position of the first AD data in the data buffer at the time calling `WD_AI_AsyncClear()`.

AccessCnt:

when pre-/middle trigger acquisition mode is not used, AccessCnt returns the number of A/D data that has been transferred at the time calling WD_AI_AsyncClear(). If double-buffered mode is enabled, AccessCnt returns the next position after the position the last A/D data is stored in the circular buffer.

Return Code

```
NoError  
ErrorInvalidCardNumber  
ErrorCardNotRegistered  
ErrorFuncNotSupport
```

WD_AI_AsyncDblBufferHalfReady

Determines whether the next half buffer of data in circular buffer is ready for transfer during an asynchronous double-buffered analog input operation.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_AsyncDblBufferHalfReady (U16 Card-  
Number, BOOLEAN *HalfReady, BOOLEAN *StopFlag)
```

Visual Basic

```
WD_AI_AsyncDblBufferHalfReady(ByVal CardNum-  
ber As Integer, HalfReady As Byte, StopFlag As  
Byte) As Integer
```

Parameters

CardNumber:

ID of the card performing asynchronous double-buffered operation.

HalfReady:

Tells whether the next half buffer of data is available. Constants TRUE and FALSE are defined in WD-DASK.h.

StopFlag:

Tells whether the asynchronous analog input operation is completed. If `StopFlag = TRUE`, the analog input operation stopped. If `StopFlag = FALSE`, the operation is not yet completed. Constants `TRUE` and `FALSE` are defined in `WD-DASK.h`.

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
```

WD_AI_AsyncDblBufferHandled

Notifies WD-DASK that the ready buffer has been handled in user applications.

With WD-DASK, data is directly transferred through DMA to the user's buffer. Therefore, while half the buffer of data is ready (using `WD_AI_AsyncDblBufferHalfReady` to check the ready status), the data in the ready buffer can be handled directly and doesn't need to be copied to another transfer buffer. This mechanism eliminates the time taken for memory to copy and another memory space for data transfer. However, WD-DASK doesn't know if the data in the ready buffer has been handled (in a user application). If the data was handled, the user application needs an interface to notify WD-DASK about this. The new `WD_AI_AsyncDblBufferHandled` function is used for this purpose.

Supported Cards

9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_AsyncDblBufferHandled (U16 CardNumber)
```

Visual Basic

```
WD_AI_AsyncDblBufferHandled (ByVal CardNumber  
As Integer) As Integer
```

Parameter

CardNumber:

ID of the card performing asynchronous double buffered operation.

Return Code

NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport

WD_AI_AsyncDblBufferMode

Enables or disables double-buffered data acquisition mode. The double-buffer mode of continuous A/D conversions is only supported for post-trigger and delay trigger mode operation.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_AsyncDblBufferMode (U16 CardNumber,  
    BOOLEAN Enable)
```

Visual Basic

```
WD_AI_AsyncDblBufferMode (ByVal CardNumber As  
    Integer, ByVal Enable As Byte) As Integer
```

Parameters

CardNumber:

ID of the card where double-buffered mode is set.

Enable:

Indicates whether the double-buffered mode is enabled. Constants TRUE and FALSE are defined in wd-dask.h.

TRUE	Double-buffered mode is enabled.
------	----------------------------------

FALSE	Double-buffered mode is disabled.
-------	-----------------------------------

Return Code

NoError
 ErrorInvalidCardNumber
 ErrorCardNotRegistered
 ErrorFuncNotSupport



NOTE:

When performing double buffer mode of an AI operation for PCI/PXI-9820, the data is transferred to the user buffer on the system directly and not through onboard SDRAM.

WD_AI_AsyncDblBufferOverrun

Checks or clears overrun status of a double-buffered analog input operation.

Supported Cards

9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_AsyncDblBufferOverrun (U16 wCardNum-
ber, U16 op, U16 *overrunFlag)
```

Visual Basic

```
WD_AI_AsyncDblBufferOverrun (ByVal CardNumber
As Integer, ByVal op As Integer, overrunFlag
As Integer ) As Integer
```

Parameters

CardNumber:

ID of the card performing asynchronous double buffered operation.

op:

check/clear overrun status/flag.

0	Check the overrun status
1	Clear the overrun flag

overflowFlag:

Returned overrun status

0	No overrun occurs
1	Overflow occurs

Return Code

NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport

WD_AI_AsyncDblBufferToFile

If continuous AI function is:

WD_AI_ContReadChannelToFile
WD_AI_ContReadMultiChannelsToFile, and
WD_AI_ContScanChannelsToFile)

This function logs the data of the circular buffer into a disk file.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_AsyncDblBufferToFile (U16 CardNum-  
ber)
```

Visual Basic

```
WD_AI_AsyncDblBufferToFile (ByVal CardNumber  
As Integer) As Integer
```

Parameter

CardNumber:

ID of the card where double-buffered mode is set.

Return Code

NoError
ErrorInvalidCardNumber

ErrorCardNotRegistered
ErrorFuncNotSupport

WD_AI_AsyncReStartNextReady

Checks the restart mode status of continuous AI. This function is valid only for post trigger or delay trigger mode of restarted continuous AI operation.

Supported Cards.

9820, 9816, 9826, 9846

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_AsyncReStartNextReady (U16 CardNum-
ber, BOOLEAN *DaqReady, BOOLEAN *StopFlag, U16
*RdyDaqCnt)
```

Visual Basic

```
WD_AI_AsyncReStartNextReady (ByVal CardNumber
As Integer, DaqReady As Byte, StopFlag As
Byte, RdyDaqCnt As Integer) As Integer
```

Parameters

CardNumber:

ID of the card performing asynchronous double-buffered operation.

DaqReady:

Indicates whether the next data conversion is completed. Constants TRUE and FALSE are defined in WD-DASK.h.

StopFlag:

Indicates whether the definite count of restart mode of data conversion is completed. If StopFlag = TRUE, the analog input operation stopped. If StopFlag = FALSE, the operation is not yet completed. Constants TRUE and FALSE are defined in WD-DASK.h.

RdyDaqCnt:

If restart count is definite, this argument returns the count of finished data acquisition. If the restart count is infinite, this argument returns the index of the buffer that stores the most recent data.

Return Code

```
NoError  
ErrorInvalidCardNumber  
ErrorCardNotRegistered  
ErrorFuncNotSupport
```

WD_AI_AsyncReTrigNextReady

Determines whether the data associated with the next trigger signal is ready during an asynchronous re-triggered analog input operation, based on conditions in which WD_AI_Config is called prior to WD_AI_ContBufferSetup, and Asynchronous mode uses ASYNCH_OP for continuous AI operation.

Supported Cards

9842, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_AsyncReTrigNextReady (U16 CardNum-  
ber, BOOLEAN *trgReady, BOOLEAN *StopFlag, U32  
*RdyTrigCnt)
```

Visual Basic

```
WD_AI_AsyncReTrigNextReady (ByVal CardNumber  
As Integer, trgReady As Byte, StopFlag As  
Byte, RdyTrigCnt As Long) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation

trgReady:

Reports on availability of data associated with the next trigger signal, with constants TRUE and FALSE defined in WD-DASK64.H.

StopFlag:

Reports on completion of asynchronous analog input, wherein if StopFlag = TRUE, analog input has stopped, and if StopFlag = FALSE, the operation has not, with constants TRUE and FALSE defined in WD-DASK.h / WD_DASK64.H.

RdyTrigCnt:

Returns the trigger signal count occurring if re-trigger count is definite, and if re-trigger count is infinite, returns the index of the buffer storing data after generation of the most recent trigger signal trigger.

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
```

WD_AI_CH_ChangParam

Resets WD-DASK library of the AI range, FIR filter, impedance, or ADC dither for the specified channel of the card with card ID CardNumber.

Cards Support

9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++ and Borland C++

```
I16 WD_AI_CH_ChangeParam (U16 CardNumber, U16
Channel, U16 ParamID, U16 Value);
```

Visual Basic

```
WD_AI_CH_ChangeParam (ByVal CardNumber As
Integer, ByVal Channel As Integer, ByVal ParamID As Integer, ByVal Value As Integer) As Integer
```

Parameters

CardNumber:

ID of the card to perform this operation.

Channel:

The A/D channel to perform the channel setting. Valid values are:

9816/9826/9846	0 through 3 or All_Channels (-1)
9848	0 through 7 or All_Channels (-1)
9852	0, 1 or All_Channels (-1)
9814	0 through 3 or All_Channels (-1)

ParamID:

The AD channel parameter to be changed. Valid values are:

AI_RANGE
AI_IMPEDANCE
ADC_DITHER
AI_COUPLING
AI_Bandwidth
SIGNAL_FIR

Value:

The setting for the parameter for the specified by ParamID.

AI_RANGE	See "AI Range Codes" on page 114, for valid range values
AI_IMPEDANCE	IMPEDANCE_50Ohm (50 Ohm)
	IMPEDANCE_HI (1M Ohm)
ADC_DITHER	ADC_DITHER_DIS
	ADC_DITHER_EN
AI_COUPLING	DC_Coupling
	AC_Coupling
AI_Bandwidth	BANDWIDTH_DEVICE_DEFAULT
	BANDWIDTH_20M
	BANDWIDTH_100M

SIGNAL_FIR	FIR_DIS FIR_EN_20M FIR_EN_10M
------------	-------------------------------------

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
```

WD_AI_CH_Config

Notifies the WD-DASK library of the AI range selected for the specified channel of the card with card ID CardNumber. After the WD_Register_Card function is called, all analog input channels are configured as AD_B_5_V for PCI/PXI-9820, AD_B_1_V for PCI/PXI-9816D/26D/46D/PXIe-9842, AD_B_5_V for PCI/PXI-9816H/26H/46H, AD_B_2_V for PXIe-9848/PCIe/PXIe-9852 and AD_B_10_V for PCI/PXI-9816V/26V/46V and PCIe-9814 by default. If you want to use the device with the default settings, there is no need to call this function again to configure the channel(s). Otherwise, you need to call this function to program the device for specified settings before calling any function to perform analog input operation.

Supported Cards

9820, 9816, 9826, 9846, 9842, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_CH_Config (U16 CardNumber, U16 Channel, U16 AdRange)
```

Visual Basic

```
WD_AI_CH_Config (ByVal CardNumber As Integer,
ByVal Channel As Integer, ByVal AdRange As Integer) As Integer
```

Parameters

CardNumber.

ID of the card performing the operation.

Channel:

The A/D channel to perform the channel setting. Valid range is 0 to 7 or All_Channels (-1).

AdRange:

Analog input channel settings. This argument is an integer expression formed from one or more of the manifest constants defined in WD-DASK.h. We define some constants to represent various A/D input ranges in WD-DASK.h. Refer to **Appendix A.2: AI Range Codes** for the valid range values. The default setting is AD_B_5_V for PCI/PXI-9820, AD_B_1_V for PCI/PXI-9816D/26D/46D and PCIe-9842, AD_B_5_V for PCI/PXI-9816H/26H/46H, AD_B_10_V for PCI/PXI-9816V/26V/46V/PCIe-9814, AD_B_2_V for PXIe-9848, and AD_B_2_V for PCIe/PXIe-9852.

Return Code

```
NoError  
ErrorInvalidCardNumber  
ErrorCardNotRegistered  
ErrorFuncNotSupport
```

WD_AI_Config

Informs the WD-DASK library of the timebase source, conversion source, and sampling mode for the WD-DASK device with card ID CardNumber. After calling the WD_Register_Card function, the device is configured with the following default settings:

- ▷Timebase: WD_IntTimeBase
- ▷A/D conversion source:
WD_AI_ADCONVSRC_TimePacer
- ▷A/D sampling mode: Double-edge-sampling disabled
- ▷adDutyRestore: AD duty cycle restore function enabled
- ▷Auto reset buffer: Enabled (AutoResetBuf: TRUE)

If you want use the device with the default settings, you do not need to call this function again to set the configuration.

Otherwise, this function has to be called before calling function to perform continuous analog input operation. For the PCIe-9814, if WD_ExtTimeBase or WD_DBoard_TimeBase are selected as timebase, this function must be called again whenever frequency of the timebase is changed.

Supported Cards

9820, 9816, 9826, 9846, 9842, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_Config (U16 CardNumber, U16 Time-
Base, BOOLEAN adDutyRestore, U16 ConvSrc,
BOOLEAN doubleEdged, BOOLEAN AutoResetBuf)
```

Visual Basic

```
WD_AI_Config (ByVal CardNumber As Integer,
ByVal TimeBase As Integer, ByVal adDutyRestore
As Byte, ByVal ConvSrc As Integer, ByVal dou-
bleEdged As Byte, ByVal AutoResetBuf As Byte)
As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

TimerBase:

The selected timebase. Valid values are:

WD_IntTimeBase	Internal timer is the timebase.
WD_ExtTimeBase	External timer is the timebase (NOT applicable for PCIe-9842/PXIe-9842, PXIe-9848).
WD_StarTimeBase	Clock source is from PXI_STAR line (NOT applicable for PXIe-9842, PXIe-9848).
WD_SSITimeBase	Timer is based on SSI source (NOT applicable for PCIe-9842, and PXIe-9848, and PXIe-9852 and PCIe-9814).

WD_PLL_REF_PXICLK10	Phase-locked loop (PLL) reference clock source with PXI_CLK10 as the source for the phase-locked loop. (NOT applicable for PCI/PXI-9816/26/46, and PXIe-9842)
WD_PXI_CLK10	PXI CLK is the timebase (NOT applicable for PCI/PXI-9820, PCIe-9842, PXIe-9848, and PXIe-9852).
WD_PLL_REF_EXT10	Phase-locked loop (PLL) reference clock source with an external reference clock as the source for the phase-locked loop (NOT applicable for PCI/PXI-9816/26/46, PCIe-9842, PCIe-9814, or PXIe-9848)
WD_PXIe_CLK100	PXIe 100M CLK is the timebase (ONLY available for PXIe device).
WD_PLL_REF_PXIeCLK100	Phase-locked loop (PLL) reference clock source with PXIe_CLK100 as the source for the phase-locked loop. (ONLY available for PXIe-9848, and PXIe-9852 devices).
WD_DBoard_TimeBase	Timebase signal originates with an embedded daughterboard (PCIe-9814 only)

adDutyRestore

TRUE	Activate AD duty cycle restore function. (Default)
FALSE	Deactivate AD duty cycle restore function.

ConvSrc:

A/D conversion source selection. Valid value:

WD_AI_ADCONVSRC_TimePacer	Timer pacer (Default)
---------------------------	-----------------------

doubleEdged

FALSE	Disable AD ping pong mode. (Default)
-------	--------------------------------------

AutoResetBuf

FALSE	The AI buffers set by WD_AI_ContBufferSetup are retained and WD_AI_ContBufferReset must be called to reset the buffer.
-------	--

TRUE	The AI buffers set WD_AI_ContBufferSetup are reset automatically by the driver when the AI operation is completed.
------	--

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
```

WD_AI_ContBufferReset

Resets all the buffers set by WD_AI_ContBufferSetup for continuous analog input. Call this function when the data buffers will not be used.

Supported Cards

9820, 9816, 9826, 9846, 9842, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_ContBufferReset (U16 CardNumber)
```

Visual Basic

```
WD_AI_ContBufferReset (ByVal CardNumber As Integer) As Integer
```

Parameter

CardNumber.

ID of the card performing the operation.

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
ErrorTransferCountTooLarge
ErrorContIoNotAllowed
```

WD_AI_ContBufferSetup

Sets up the buffer for continuous analog input. Call this function repeatedly to setup all data buffers (maximum of two buffers).

Supported Cards

9820, 9816, 9826, 9846, 9842, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
U16 WD_AI_ContBufferSetup (U16 CardNumber,  
void *Buffer, U32 ReadCount, U16 *BufferId)
```

Visual Basic

```
WD_AI_ContBufferSetup (ByVal CardNumber As  
Integer, Buffer As Any, ByVal ReadCount As  
Long, BufferId As Integer) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

Buffer:

Starting address of the memory to contain the input data. For PCIe-9842/PCIe-9814 and PXIe-9848, the buffer address should be 16-byte alignment, for PCIe/PXIe-9852, 8-byte alignment, and the other modules 4-byte alignment. To make sure the allocated buffer address is aligned 4 or 16, using *VirtualAlloc* (Win32 API) or *WD_Buffer_Alloc* to allocate memory.

ReadCount:

Size (in samples) of the buffer. The value must be a multiple of 8 for PCIe-9842/PXIe-9848/PCIe-9814, a multiple of 8 for PCIe/PXIe-9852, and even for the other modules.

BufferId:

Returns the index of the currently set up buffer.

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
ErrorTransferCountTooLarge
ErrorContIoNotAllowed
```

WD_AI_ContBufferSetup32

Sets up the buffer for continuous 32-bit analog input. Call this function repeatedly to set up all data buffers (maximum of two buffers).

Supported Cards

9852

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_ContBufferSetup32 (U16 CardNumber,
void *Buffer, U32 ReadCount, U16 *BufferId)
```

Visual Basic

```
WD_AI_ContBufferSetup32 (ByVal CardNumber As Integer, Buffer As Any, ByVal ReadCount As Long, BufferId As Integer) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

Buffer:

Starting address of the memory to contain the input data. To make sure the allocated buffer address is aligned 4 or 16, using VirtualAlloc (Win32 API) or WD_Buffer_Alloc to allocate memory..

ReadCount:

Size (in samples) of the buffer. The value must be even.

BufferId:

Returns the index of the currently set up buffer.

Return Code

NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
ErrorTransferCountTooLarge
ErrorInvalidAdRange

WD_AI_ContReadChannel

Performs continuous A/D conversions on the specified analog input channel at a rate as close to the rate you specified. The double buffer mode of continuous A/D conversions is only supported for Post-trigger and Delay trigger mode operation.

Supported Cards

9820, 9816, 9826, 9846, 9842, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_ContReadChannel (U16 CardNumber, U16  
Channel, U16 BufId, U32 ReadScans, U32 ScanIn-  
trv, U32 SampIntrv, U16 SyncMode)
```

Visual Basic

```
WD_AI_ContReadChannel (ByVal CardNumber As  
Integer, ByVal Channel As Integer, ByVal BufId  
As Integer, ByVal ReadScans As Long, ByVal  
ScanIntrv As Long, ByVal SampIntrv As Long,  
ByVal SyncMode As Integer) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

Channel:

Analog input channel number.

BufId:

The buffer ID (returned from function `WD_AI_ContBufferSetup`) of the buffer containing the acquired data. When two steps of data acquisition (DAQSTEPPED) is not performed, the size of the BufId buffer must have a length equal to the value of `ReadScans`. When double-buffered mode is enabled, the starting buffer id should be 0. You may ignore this argument. Refer to **Appendix A.3: AI Data Format** for the BufId data format.

ReadScans:

When double-buffered mode is disabled, shows the total number of scans to be performed, with restrictions for each module as follows.

Module	Restrictions
PCI/PXI-9820	▶ <i>ReadScans</i> must be a multiple of 2 ▶ The total data bytes may not exceed the size of the onboard SDRAM ▶ In pre-trigger and middle-trigger modes, <i>ReadScans</i> must be a multiple of 32
PCI/PXI-9816/26/46	▶ <i>ReadScans</i> must be a multiple of 2 ▶ In DAQSTEPPED mode (See “WD_AI_Set_Mode” on page 68), total data bytes may not exceed the size of the onboard SDRAM ▶ In pre-trigger and middle-trigger modes, total data bytes may not exceed the size of the onboard SDRAM
PCIe-9842	<i>ReadScans</i> must be a multiple of 8
PXIe-9848	▶ <i>ReadScans</i> must be a multiple of 8. ▶ In DAQSTEPPED mode (See “WD_AI_Set_Mode” on page 68), <i>ReadScans</i> may not exceed 32M ▶ In pre-trigger and middle-trigger modes, <i>ReadScans</i> may not exceed 32M

Module	Restrictions
PCIe/PXle-9852	▶ <i>ReadScans</i> must be a multiple of 8 ▶ In DAQSTEPPED mode (refer to <i>WD_AI_Set_Mode</i>), <i>ReadScans</i> may not exceed 256M ▶ In pre-trigger and middle-trigger modes, <i>ReadScans</i> may not exceed 256M ▶ In Average mode, <i>ReadScans</i> may not exceed 128M
PCIe-9814	▶ <i>ReadScans</i> must be a multiple of 8 ▶ For DAQSTEPPED mode (See “<i>WD_AI_Set_Mode</i>” on page 68), <i>ReadScans</i> may not exceed the size of the onboard SDRAM (512M) ▶ For pre-trigger and middle-trigger mode operation, the <i>ReadScans</i> may not exceed 512M

Table 2-1: WD_AI_ContReadChannel ReadScan Module Restrictions

For double-buffered acquisition, *ReadScans* is the size (in samples) allocated for each channel in the circular buffer. This value must be a multiple of two.



NOTE:

For pre-trigger and middle trigger AI modes, *ReadScans* must equal the sum of *postTrigScans*+*preTrigScans*.

ScanIntrv:

Length of scan interval or the counter value between the initiation of each scan sequence. The scan rate is $\text{TimeBase}/\text{ScanIntrv}$. The valid range is 1 to 65535 for PCIe-9842, PXle-9848, PCIe/PXle-9852 and PCIe-9814 and 1 to 16777215 for the other modules.

SamplIntrv:

Length of the sample interval or the counter value between each A/D conversion within a scan sequence. The A/D conversion rate is $\text{TimeBase}/\text{SamplIntrv}$. The value of *TimeBase* depends on the card type. The valid range is 1 to

65535 for PCIe-9842/PCIe/PXIe-9852/PCIe-9814, and 1 to 16777215 for other modules..



NOTE:

For simultaneous AD cards, such as PCI/PXI-9820, PCI/PXI-9816/26/46, PCIe-9842/PCIe/PXIe-9852/PCIe-9814 and PXIe-9848, SampIntrv has no function.

SyncMode:

Tells whether the operation is performed synchronously or asynchronously. If pre-/middle trigger mode is enabled by calling the `WD_AI_Trig_Config()` function, this operation should be performed asynchronously. Valid values are:

SYNCH_OP	Synchronous A/D conversion. The function does not return until the A/D operation is completed.
ASYNCH_OP	Asynchronous A/D conversion.

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
ErrorInvalidIoChannel
ErrorInvalidAdRange
ErrorTransferCountTooLarge
ErrorContIoNotAllowed
ErrorInvalidSampleRate
```

WD_AI_ContReadChannelToFile

Performs continuous A/D conversions on the specified analog input channel at a rate closest to the specified rate, then saves the acquired data in a disk file. The data is written to disk in binary format with the lower byte first (little endian). Refer to **Appendix A.4: Data File Format** for the data file structure and **Appendix A.3: AI Data Format** for the data format of the data file. The double-buffer mode of continuous A/D conversions is only supported for post-trigger and delay trigger mode operations.

Supported Cards

9820, 9816, 9826, 9846, 9842, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_ContReadChannelToFile (U16 CardNum-  
ber, U16 Channel, U16 BufId, U8 *FileName, U32  
ReadScans, U32 ScanIntrv, U32 SampIntrv, U16  
SyncMode)
```

Visual Basic

```
WD_AI_ContReadChannelToFile (ByVal CardNumber  
As Integer, ByVal Channel As Integer, ByVal  
BufId As Integer, ByVal FileName As String,  
ByVal ReadScans As Long, ByVal ScanIntrv As  
Long, ByVal SampIntrv As Long, ByVal SyncMode  
As Integer) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

Channel:

Analog input channel number.

BufId:

The buffer ID (returned from function WD_AI_ContBufferSetup) of the buffer containing the acquired data. When two steps of data acquisition (DAQSTEPPED) is not performed, the size of the BufId buffer must have a length equal to the value of ReadScans. When double-buffered mode is enabled, the starting buffer id should be 0. You may ignore this argument. Refer to **Appendix A.3: AI Data Format** for the BufId data format.

FileName:

Name of data file that stores the acquired data.

ReadScans:

When double-buffered mode is disabled, shows the total number of scans to be performed, with restrictions for each module as follows.

Module	Restrictions
PCI/PXI-9820	▶ <i>ReadScans</i> must be a multiple of 2 ▶ The total data bytes may not exceed the size of the onboard SDRAM ▶ In pre-trigger and middle-trigger modes, <i>ReadScans</i> must be a multiple of 32
PCI/PXI-9816/26/46	▶ <i>ReadScans</i> must be a multiple of 2 ▶ In DAQSTEPPED mode (See “WD_AI_Set_Mode” on page 68), total data bytes may not exceed the size of the onboard SDRAM ▶ In pre-trigger and middle-trigger modes, total data bytes may not exceed the size of the onboard SDRAM
PCIe-9842	<i>ReadScans</i> must be a multiple of 8
PXIe-9848	▶ <i>ReadScans</i> must be a multiple of 8. ▶ In DAQSTEPPED mode (See “WD_AI_Set_Mode” on page 68), <i>ReadScans</i> may not exceed 32M ▶ In pre-trigger and middle-trigger modes, <i>ReadScans</i> may not exceed 32M
PCIe/PXIe-9852	▶ <i>ReadScans</i> must be a multiple of 8 ▶ In DAQSTEPPED mode (refer to WD_AI_Set_Mode), <i>ReadScans</i> may not exceed 256M ▶ In pre-trigger and middle-trigger modes, <i>ReadScans</i> may not exceed 256M ▶ In Average mode, <i>ReadScans</i> may not exceed 128M

Module	Restrictions
PCIe-9814	▶ ReadScans must be a multiple of 8 ▶ For DAQSTEPPED mode (See “WD_AI_Set_Mode” on page 68), ReadScans may not exceed the size of the onboard SDRAM (512M) ▶ For pre-trigger and middle-trigger mode operation, the ReadScans may not exceed 512M

Table 2-2: WD_AI_ContReadChannelToFile ReadScan Module Restrictions

For double-buffered acquisition, ReadScans is the size (in samples) allocated for each channel in the circular buffer. This value must be a multiple of two..



NOTE:

For pre-trigger and middle trigger AI modes, *ReadScans* must equal the sum of *postTrigScans*+*preTrigScans*.

ScanIntrv:

Length of scan interval or the counter value between the initiation of each scan sequence. The scan rate is $\text{TimeBase}/\text{ScanIntrv}$. The value of TimeBase depends on the card type. The valid range is 1 to 65535 for PCIe-9842, PXIe-9848, PCIe/PXIe-9858, and PCIe-9814, and 1 to 16777215 for the other modules.

SamplIntrv:

Length of the sample interval or the counter value between each A/D conversion within a scan sequence. The A/D conversion rate is $\text{TimeBase}/\text{SamplIntrv}$. The value of TimeBase depends on the card type. The valid range is 1 to 65535 for PXIe-9842/9848 and 1 to 16777215 for the other modules.



NOTE:

For simultaneous AD cards, such as PCI/PXI-9820, PCI/PXI-9816/26/46, PCIe-9842,, PCIe/PXIe-9858, PCIe-9814, and PXIe-9848, *SamplIntrv* has no function.

SyncMode:

Tells whether the operation is performed synchronously or asynchronously. If pre-/middle trigger mode is enabled by calling the `WD_AI_Trig_Config()` function, this operation should be performed asynchronously. Valid values are:

SYNCH_OP	Synchronous A/D conversion. The function does not return until the A/D operation is completed.
ASYNCH_OP	Asynchronous A/D conversion.

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
ErrorInvalidIoChannel
ErrorInvalidAdRange
ErrorTransferCountTooLarge
ErrorContIoNotAllowed
ErrorInvalidSampleRate
ErrorOpenFile
```

WD_AI_ContReadMultiChannels

Performs continuous A/D conversions on the specified analog input channels at a rate closest to the specified rate. The double-buffer mode of continuous A/D conversions is only supported for post-trigger and delay trigger mode operations.

Supported Cards

9820, 9816, 9826, 9846, 9842, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_ContReadMultiChannels (U16 CardNumber,
U16 NumChans, U16 *Chans, U16 BufId, U32
ReadScans, U32 ScanIntrv, U32 SampIntrv, U16
SyncMode)
```

Visual Basic

```
WD_AI_ContReadMultiChannels (ByVal CardNumber
As Integer, ByVal NumChans As Integer, Chans
```

```
As Integer, ByVal BufId As Integer, ByVal  
ReadScans As Long, ByVal ScanIntrv As Long,  
ByVal SampIntrv As Long, ByVal SyncMode As  
Integer) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

NumChans:

Number of analog input channels in the array Chans. Valid value is 1 to 8.

Chans:

Array of analog input channel numbers. For PCI/PXI-9820, the numbers in Chans must be within 0 and 1 and for PCIe/PXle-9852, the numbers in Chans must be within 0 and 1, for PCIe-9842 the number must be 0 and for PCI/PXI-9816/26/46 and PCIe-9814, the numbers in Chans must be within 0 and 3 and for PXle-9848, the numbers in Chans must be within 0 and 7.

BufId:

The buffer ID (returned from function WD_AI_ContBufferSetup) of the buffer containing the acquired data. When two steps of data acquisition (DAQSTEPPED) is not performed, the size of the BufId buffer must have a length equal to the value of *ReadScans*. When double-buffered mode is enabled, the starting buffer id should be 0. You may ignore this argument. Refer to **Appendix A.3: AI Data Format** for the BufId data format.

ReadScans:

When double-buffered mode is disabled, shows the total number of scans to be performed, with restrictions for each module as follows.

Module	Restrictions
PCI/PXI-9820	▶ <i>ReadScans</i> must be a multiple of 2 ▶ Total data bytes may not exceed the size of the onboard SDRAM ▶ In pre-trigger and middle-trigger modes, <i>ReadScans</i> must be a multiple of 32
PCI/PXI-9816/26/46	▶ <i>ReadScans</i> must be a multiple of 2. ▶ In DAQSTEPPED mode (See "WD_AI_Set_Mode" on page 68), total data bytes may not exceed the size of the onboard SDRAM ▶ In pre-trigger and middle-trigger modes, total data bytes may not exceed the size of the onboard SDRAM
PCIe-9842	<i>ReadScans</i> must be a multiple of 8
PXIe-9848	▶ <i>ReadScans</i> must be a multiple of 8. ▶ In DAQSTEPPED mode (refer to WD_AI_Set_Mode), <i>ReadScans</i> may not exceed 32M ▶ In pre-trigger and middle-trigger modes, <i>ReadScans</i> may not exceed 32M
PCIe/PXIe-9852	▶ <i>ReadScans</i> must be a multiple of 8 ▶ In DAQSTEPPED mode (refer to WD_AI_Set_Mode), <i>ReadScans</i> may not exceed 256M ▶ In pre-trigger and middle-trigger modes, <i>ReadScans</i> may not exceed 256M ▶ In Average mode, <i>ReadScans</i> may not exceed 128M

Module	Restrictions
PCle-9814	▶ The ReadScans * NumChans must be a multiple of 8. ▶ For DAQSTEPPED mode (See "WD_AI_Set_Mode" on page 68), the ReadScans may not exceed 512M ▶ For pre-trigger and middle-trigger mode operation, ReadScans may not exceed 512M

Table 2-3: WD_AI_ContReadMultiChannels ReadScan Module Restrictions

For double-buffered acquisition, ReadScans is the size (in samples) allocated for each channel in the circular buffer. This value must be a multiple of two.



NOTE:

For pre-trigger and middle trigger AI modes, ReadScans must equal postTrigScans+preTrigScans.

ScanIntrv:

Length of scan interval or the counter value between the initiation of each scan sequence. The scan rate is TimeBase/ScanIntrv. The valid range is 1 to 65535 for PCle-9842, PXle-9848, PCle/PXle-9852, and PCle-9814 and 1 to 16777215 for the other modules.

SamplIntrv:

Length of the sample interval or the counter value between each A/D conversion within a scan sequence. The A/D conversion rate is TimeBase/SamplIntrv. The value of TimeBase depends on the card type. The valid range is 1 to 65535 for PXle-9842/PXle-9848 and 1 to 16777215 for the other modules.



NOTE:

For simultaneous AD cards, such as PCI/PXI-9820, PCI/PXI-9816/26/46, PCle-9842, PCle/PXle-9852, PCle-9814, and PXle-9848, SamplIntrv has no function.

SyncMode:

Tells whether the operation is performed synchronously or asynchronously. If pre-/middle trigger mode is enabled by calling the `WD_AI_Trig_Config()` function, this operation should be performed asynchronously. Valid values are:

SYNCH_OP	Synchronous A/D conversion. The function does not return until the A/D operation is completed.
ASYNCH_OP	Asynchronous A/D conversion.

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
ErrorInvalidIoChannel
ErrorInvalidSampleRate
ErrorInvalidAdRange
ErrorTransferCountTooLarge
ErrorContIoNotAllowed
```

WD_AI_ContReadMultiChannelsToFile

Performs continuous A/D conversions on the specified analog input channels at a rate closest to the specified rate, then saves the acquired data in a disk file. The data is written to disk in binary format with the lower byte first (little endian). Refer to **Section A.4 Data File Format** for the data file structure and format. The double-buffer mode of continuous A/D conversions is only supported for post-trigger and delay trigger mode operations.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16    WD_AI_ContReadMultiChannelsToFile    (U16
CardNumber, U16 NumChans, U16 *Chans, U16
BufId, U8 *FileName, U32 ReadScans, U32
ScanIntrv, U32 SampIntrv, U16 SyncMode)
```

Visual Basic

```
WD_AI_ContScanChannelsToFile (ByVal CardNum-  
ber As Integer, ByVal NumChans As Integer,  
Chans As Integer, ByVal FileName As String,  
ByVal ReadScans As Long, ByVal ScanIntrv As  
Long, ByVal SampIntrv As Long, ByVal SyncMode  
As Integer) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

NumChans:

Number of analog input channels in the array Chans. Valid value is 1 to 8.

Chans:

Array of analog input channel numbers. For PCI/PXI-9820 and PCIe/PXle-9852, the numbers in Chans must be within 0 and 1, for PXle-9848 the numbers in Chans must be within 0 and 7, for PCIe-9842 the number must be 0 and for PCI/PXI-9816/26/46 and PCIe-9814, the numbers in Chans must be within 0 and 3.

BufId:

The buffer ID (returned from function WD_AI_ContBufferSetup) of the buffer containing the acquired data. When two steps of data acquisition (DAQSTEPPED) is not performed, the size of the BufId buffer must have a length equal to the value of *ReadScans*. When double-buffered mode is enabled, the starting buffer id should be 0. You may ignore this argument. Refer to **Appendix A.3: AI Data Format** for the BufId data format.

FileName:

Name of data file that stores the acquired data.

ReadScans:

When double-buffered mode is disabled, shows the total number of scans to be performed, with restrictions for each module as follows.

Module	Restrictions
PCI/PXI-9820	▶ <i>ReadScans</i> must be a multiple of 2 ▶ Total data bytes may not exceed the size of the onboard SDRAM ▶ In pre-trigger and middle-trigger modes, <i>ReadScans</i> must be a multiple of 32
PCI/PXI-9816/26/46	▶ <i>ReadScans</i> must be a multiple of 2. ▶ In DAQSTEPPED mode (See “WD_AI_Set_Mode” on page 68), total data bytes may not exceed the size of the onboard SDRAM ▶ In pre-trigger and middle-trigger modes, total data bytes may not exceed the size of the onboard SDRAM
PCIe-9842	<i>ReadScans</i> must be a multiple of 8
PXIe-9848	▶ <i>ReadScans</i> must be a multiple of 8. ▶ In DAQSTEPPED mode (refer to WD_AI_Set_Mode), <i>ReadScans</i> may not exceed 32M ▶ In pre-trigger and middle-trigger modes, <i>ReadScans</i> may not exceed 32M
PCIe/PXIe-9852	▶ <i>ReadScans</i> must be a multiple of 8 ▶ In DAQSTEPPED mode (refer to WD_AI_Set_Mode), <i>ReadScans</i> may not exceed 256M ▶ In pre-trigger and middle-trigger modes, <i>ReadScans</i> may not exceed 256M ▶ In Average mode, <i>ReadScans</i> may not exceed 128M
PCIe-9814	▶ The <i>ReadScans</i> * NumChans must be a multiple of 8. ▶ For DAQSTEPPED mode (See “WD_AI_Set_Mode” on page 68), the <i>ReadScans</i> may not exceed 512M ▶ For pre-trigger and middle-trigger mode operation, <i>ReadScans</i> may not exceed 512M

Table 2-4: WD_AI_ContReadMultiChannelsToFile ReadScan Module Restrictions

For double-buffered acquisition, ReadScans is the size (in samples) allocated for each channel in the circular buffer. This value must be a multiple of two.



NOTE:

For pre-trigger and middle trigger AI modes, *ReadScans* must equal *postTrigScans*+*preTrigScans*.

ScanIntrv:

Length of scan interval or the counter value between the initiation of each scan sequence. The scan rate is $\text{TimeBase}/\text{ScanIntrv}$. The value of TimeBase depends on the card type. The valid range is 1 to 65535 for PCIe-9842, PXIe-9848, PCIe/PXIe-9852, and PCIe-9814, and 1 to 16777215 for the other modules.

SamplIntrv:

Length of the sample interval or the counter value between each A/D conversion within a scan sequence. The A/D conversion rate is $\text{TimeBase}/\text{SamplIntrv}$. The value of TimeBase depends on the card type. The valid range is 1 to 65535 for PXIe-9842/PXIe-9848 and 1 to 16777215 for the other modules.



NOTE:

For simultaneous AD cards, such as PCI/PXI-9820, PCI/PXI-9816/26/46, PCIe-9842, PCIe/PXIe-9852, PCIe-9814, and PXIe-9848, *SamplIntrv* has no function.

SyncMode:

Tells whether the operation is performed synchronously or asynchronously. If pre-/middle trigger mode is enabled by calling the `WD_AI_Trig_Config()` function, this operation should be performed asynchronously. Valid values are:

SYNCH_OP	Synchronous A/D conversion. The function does not return until the A/D operation is completed.
ASYNCH_OP	Asynchronous A/D conversion.

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
ErrorInvalidIoChannel
ErrorInvalidSampleRate
ErrorInvalidAdRange
ErrorTransferCountTooLarge
ErrorContIoNotAllowed
ErrorOpenFile
```

WD_AI_ContScanChannels

Performs continuous A/D conversions on the specified continuous analog input channels at a rate closest to the specified rate. This function takes advantage of the card's simultaneous or auto-scan functionality to perform multi-channel analog input. The double buffer mode of continuous A/D conversions is only supported for post-trigger and delay trigger mode operations.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_ContScanChannels (U16 CardNumber,
U16 Channel, U16 BufId, U32 ReadScans, U32
ScanIntrv, U32 SampIntrv, U16 SyncMode)
```

Visual Basic

```
WD_AI_ContScanChannels (ByVal CardNumber As
Integer, ByVal Channel As Integer, ByVal BufId
As Integer, ByVal ReadScans As Long, ByVal
ScanIntrv As Long, ByVal SampIntrv As Long,
ByVal SyncMode As Integer) As Integer
```

Parameters

CardNumber.

ID of the card performing the operation.

Channel:

The largest channel number of specified continuous analog input channel. The number of Channel must be within 0 and 1 for PCI/PXI-9820 and PCIe/PXIe-9852, the number must be 0 for PXIe-9842, the number must be within 0 and 3 for PCI/PXI-9816/26/46 and PCIe-9814 and the number must be within 0 and 7 for PXIe-9848, in order to acquire data. The continuous scan sequence is ascending and the first one must be zero. For example:

0, 1.

BufId:

The buffer ID (returned from function `WD_AI_ContBufferSetup`) of the buffer containing the acquired data. When two steps of data acquisition (DAQSTEPPED) is not performed, the size of the BufId buffer must have a length equal to the value of `ReadScans * (number of channels per scan)`. When double-buffered mode is enabled, the starting buffer id should be 0. You may ignore this argument. Refer to **Appendix A.3: AI Data Format** for the BufId data format.

ReadScans:

When double-buffered mode is disabled, shows the total number of scans to be performed, with restrictions for each module as follows.

Module	Restrictions
PCI/PXI-9820	▶ <code>ReadScans * (number of channels per scan)</code> must be a multiple of 2 ▶ Total data bytes may not exceed the size of the onboard SDRAM ▶ In pre-trigger and middle-trigger modes, the <code>ReadScans * (number of channels per scan)</code> must be a multiple of 32

Module	Restrictions
PCI/PXI-9816/26/46	► ReadScans * (number of channels per scan) must be a multiple of 2 ► In DAQSTEPPED mode (See "WD_AI_Set_Mode" on page 68), total data bytes may not exceed the size of the onboard SDRAM ► In pre-trigger and middle-trigger modes, the total data bytes may not exceed the size of the onboard SDRAM
PCIe-9842	► ReadScans * (number of channels per scan) must be a multiple of 8
PXIe-9848	► ReadScans * (number of channels per scan) must be a multiple of 8 ► In DAQSTEPPED mode (See "WD_AI_Set_Mode" on page 68), the ReadScans may not exceed 32M ► In pre-trigger and middle-trigger modes, the ReadScans may not exceed 32M
PCIe/PXIe-9852	► ReadScans * (number of channels per scan) must be a multiple of 8 ► In DAQSTEPPED mode (See "WD_AI_Set_Mode" on page 68), the ReadScans may not exceed 256M. ► In pre-trigger and middle-trigger modes, the ReadScans may not exceed 256M ► In Average mode, the ReadScans may not exceed 128M
PCIe-9814	► The ReadScans * NumChans must be a multiple of 8. ► For DAQSTEPPED mode (See "WD_AI_Set_Mode" on page 68), the ReadScans may not exceed 512M ► For pre-trigger and middle-trigger mode operation, ReadScans may not exceed 512M

Table 2-5: WD_AI_ContScanChannels ReadScan Module Restrictions

For double-buffered acquisition, ReadScans is the size (in samples) allocated for each channel in the circular buffer. This value must be a multiple of two.



NOTE:

For pre-trigger and middle trigger AI modes, *ReadScans* must equal *postTrigScans*+*preTrigScans*.

ScanIntrv:

Length of scan interval or the counter value between the initiation of each scan sequence. The scan rate is $\text{TimeBase}/\text{ScanIntrv}$. The value of *TimeBase* depends on the card type. The valid range is 1 to 65535 for PCIe-9842, PXIe-9852 and PCIe-9814, and 1 to 16777215 for the other modules.

SamplIntrv:

Length of the sample interval or the counter value between each A/D conversion within a scan sequence. The A/D conversion rate is $\text{TimeBase}/\text{SamplIntrv}$. The value of *TimeBase* depends on the card type. The valid range is 1 to 65535 for PCIe-9842 and 1 to 16777215 for the other modules.



NOTE:

For simultaneous AD cards, such as PCI/PXI-9820, PCI/PXI-9816/26/46, PCIe-9842, PCIe/PXIe-9852, PCIe-9814, and PXIe-9848, *SamplIntrv* has no function.

SyncMode:

Tells whether the operation is performed synchronously or asynchronously. If pre-/middle trigger mode is enabled by calling the `WD_AI_Trig_Config()` function, this operation should be performed asynchronously. Valid values are:

SYNCH_OP	Synchronous A/D conversion. The function does not return until the A/D operation is completed.
ASYNCH_OP	Asynchronous A/D conversion.

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
ErrorInvalidIoChannel
ErrorInvalidSampleRate
ErrorInvalidAdRange
ErrorTransferCountTooLarge
ErrorContIoNotAllowed
ErrorLastChannelNotZero
ErrorDiffRangeNotSupport
ErrorChannelNotDescending
ErrorChannelNotAscending
```

WD_AI_ContScanChannelsToFile

Performs continuous A/D conversions on the specified continuous analog input channels at a rate closest to the specified rate, then saves the acquired data in a disk file. The data is written to disk in binary format with the lower byte first (little endian). Refer to **Appendix A.4: Data File Format** for the data file structure and **Appendix A.3: AI Data Format** for the data format of the data file.

This function takes advantage of the card's simultaneous or auto-scan functionality to perform multi-channel analog input. The double buffer mode of continuous A/D conversions is only supported for post-trigger and delay trigger mode operations.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_ContScanChannelsToFile (U16 CardNum-
ber, U16 Channel, U16 BufId, U8 *FileName, U32
ReadScans, U32 ScanIntrv, U32 SampIntrv, U16
SyncMode);
```

Visual Basic

```
WD_AI_ContScanChannelsToFile (ByVal CardNum-  
ber As Integer, ByVal Channel As Integer,  
ByVal BufId As Integer, ByVal FileName As  
String, ByVal ReadScans As Long, ByVal ScanIn-  
trv As Long, ByVal SampIntrv As Long, ByVal  
SyncMode As Integer) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

Channel:

The largest channel number of specified continuous analog input channel. The number of Channel must be within 0 and 1 for PCI/PXI-9820 and PCIe/PXle-9852, within 0 and 7 for PXle-9848, for PCIe-9842 the number must be 0 and within 0 and 3 for PCI/PXI-9816/26/46 and PCIe-9814 and PCIe-9852 in order to acquire data. The continuous scan sequence is ascending and the first one must be zero. For example: 0, 1.

BufId:

The buffer ID (returned from function WD_AI_ContBufferSetup) of the buffer containing the acquired data. When two steps of data acquisition (DAQSTEPPED) is not performed, the size of the BufId buffer must have a length equal to the value of ReadScans. When double-buffered mode is enabled, the starting buffer id should be 0. You may ignore this argument. Refer to **Appendix A.3: AI Data Format** for the BufId data format.

FileName:

Name of data file that stores the acquired data.

ReadScans::

When double-buffered mode is disabled, shows the total number of scans to be performed, with restrictions for each module as follows.

Module	Restrictions
PCI/PXI-9820	► ReadScans * (number of channels per scan) must be a multiple of 2 ► Total data bytes may not exceed the size of the onboard SDRAM ► In pre-trigger and middle-trigger modes, the ReadScans * (number of channels per scan) must be a multiple of 32
PCI/PXI-9816/26/46	► ReadScans * (number of channels per scan) must be a multiple of 2 ► In DAQSTEPPED mode (See “WD_AI_Set_Mode” on page 68), total data bytes may not exceed the size of the onboard SDRAM ► In pre-trigger and middle-trigger modes, the total data bytes may not exceed the size of the onboard SDRAM
PCle-9842	► ReadScans * (number of channels per scan) must be a multiple of 8
PXIe-9848	► ReadScans * (number of channels per scan) must be a multiple of 8 ► In DAQSTEPPED mode (See “WD_AI_Set_Mode” on page 68), the ReadScans may not exceed 32M ► In pre-trigger and middle-trigger modes, the ReadScans may not exceed 32M
PCle-9814	► The ReadScans * NumChans must be a multiple of 8. ► For DAQSTEPPED mode (See “WD_AI_Set_Mode” on page 68), the ReadScans may not exceed 512M ► For pre-trigger and middle-trigger mode operation, ReadScans may not exceed 512M

Table 2-6: WD_AI_ContScanChannelsToFile ReadScan Module Restrictions

For double-buffered acquisition, ReadScans is the size (in samples) allocated for each channel in the circular buffer. This value must be a multiple of two



NOTE:

For pre-trigger and middle trigger AI modes, *ReadScans* must equal *postTrigScans*+*preTrigScans*.

ScanIntrv:

Length of scan interval or the counter value between the initiation of each scan sequence. The scan rate is $\text{TimeBase}/\text{ScanIntrv}$. The value of TimeBase depends on the card type. The valid range is 1 to 65535 for PCIe-9842, PXIe-9848, and PCIe/PXIe-9814, and 1 to 16777215 for the other modules.

SamplIntrv:

Length of the sample interval or the counter value between each A/D conversion within a scan sequence. The A/D conversion rate is $\text{TimeBase}/\text{SamplIntrv}$. The value of TimeBase depends on the card type. The valid range is 1 to 65535 for PCIe-9842 and PXIe- 9848, and 1 to 16777215 for the other modules.



NOTE:

For simultaneous AD cards, such as PCI/PXI-9820, PCI/PXI-9816/26/46, PCIe-9842, PCIe/PXIe-9852, PCIe-9814, and PXIe-9848, *SamplIntrv* has no function.

SyncMode:

Tells whether the operation is performed synchronously or asynchronously. If pre-/middle trigger mode is enabled by calling the `WD_AI_Trig_Config()` function, this operation should be performed asynchronously. Valid values are:

SYNCH_OP	Synchronous A/D conversion. The function does not return until the A/D operation is completed.
ASYNCH_OP	Asynchronous A/D conversion.

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
ErrorInvalidIoChannel
ErrorInvalidSampleRate
ErrorInvalidAdRange
ErrorTransferCountTooLarge
ErrorContIoNotAllowed
ErrorLastChannelNotZero
ErrorDiffRangeNotSupport
ErrorChannelNotDescending
ErrorChannelNotAscending
```

WD_AI_ContStatus

While performing continuous A/D conversions, call this function to get the A/D status. Refer to the card documentation for AI status that the card may have.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_ContStatus (U16 CardNumber, U32
*Status)
```

Visual Basic

```
WD_AI_ContStatus (ByVal CardNumber As Integer,
Status As Long) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

Status:

The returned continuous AI status. The parameter description is listed as follows:

PCI/PXI-9820:

bit 0 ~ 1	Not used.
bit 2	1 indicates that there is no trigger signal.
bit 3	Not used.
bit 4	1 indicates that the M_counter counts to zero status.
bit 5	1 indicates that the AD duty restore function is enabled.
bit 6 ~13	Not used.
bit 14	1 indicates that the SC counter is enabled.
bit 15 ~ 17	Not used.
bit 18	1 indicates that the A/D DMA FIFO is empty.
bit 19	1 indicates that the A/D Pre-FIFO is empty.
bit 20	1 indicates that the A/D Post-FIFO is empty.
bit 21 to 28	Not used.
bit 29	Indicates that double buffer overrun has occurred (only on PCI-9820 with SubsystemID 0xA820)
bit 30	Indicates that onboard DDR2 overflow has occurred (only on PCI-9820 with SubsystemID 0xA820)
bit 31	Reserved

PCI/PXI-9816/26/46:

bit 0	Reserved.
bit 1	AD Overrun status. 1: A/D overrun. 0: A/D not overrun.
bit 2	Trigger status. 1: Trigger event occurred. 0: No trigger.
bit 3	Scan counter terminal count status. 1: Scan counter counted to 0. 0: Scan counter not complete.

bit 4	M_counter counts to zero status. 1: counts to zero. 0: not zero.
bit 5 ~ 28	Reserved.
bit 29	Double buffer overrun occurred (only available for PCI-9820 with SubsystemID 0xA820)
bit 30	Onboard SDRAM overflow occurred (only available for PCI-9820 with SubsystemID 0xA820)
bit 31	Reserved

PXIe-9842:

bit 0	indicates DMA is complete
bit 1	indicates DMA FIFO is empty
bit 2	indicates DMA FIFO is almost empty
bit 3	indicates DMA FIFO is almost full
bit 4	indicates DMA FIFO is full
bit 5~7	not used
bit 8	indicates data acquisition is in progress
bit 9 ~ 31	not used

PXIe-9848:

bit 0	indicates DMA is complete
bit 1	indicates DMA FIFO is empty
bit 2	indicates DMA FIFO is almost empty
bit 3	indicates DMA FIFO is almost full
bit 4	indicates DMA FIFO is full
bit 5~7	not used
bit 8	indicates data acquisition is in progress
Bit 9	indicates DDR2 is empty
Bit 10	indicates DDR2 is full
Bit 11	Indicates Local FIFO is empty
Bit 12	Indicates Local FIFO is full
Bit 13 ~ 28	Not used
Bit 29	Double buffer overrun occurred
Bit 30	Onboard DDR2 overflow occurred

Bit 31	not used
--------	----------

PCIe/PXle-9852:

bit 0	indicates DMA is done
bit 1	indicates DMA FIFO is empty
bit 2	indicates DMA FIFO is almost empty
bit 3	indicates DMA FIFO is almost full
bit 4	indicates DMA FIFO is full
bit 5~7	not used
bit 8	indicates data acquisition is in progress
Bit 9	indicates DDR2 is empty
Bit 10	indicates DDR2 is full
Bit 11	Indicates Local FIFO is empty
Bit 12	Indicates Local FIFO is full
Bit 13	Trigger status 1: Trigger event occurred. 0: No trigger.
Bit 14 ~ 28	Not used
Bit 29	Double buffer overrun occurred
Bit 30	Onboard DDR2 overflow occurred
Bit 31	not used

PCIe-9814:

bit 0	indicates DMA is done
bit 1	indicates DMA FIFO is empty
bit 2	indicates DMA FIFO is almost empty
bit 3	indicates DMA FIFO is almost full
bit 4	indicates DMA FIFO is full
bit 5 to 7	not used
bit 8	indicates data acquisition is in progress
bit 9	indicates DDR2 is empty
bit 10	indicates DDR2 is full
bit 11	Indicates Local FIFO overflow occurred
Bit 12 to 28	Not used
Bit 29	Double buffer overrun occurred
Bit 30	Onboard DDR2 overflow occurred
Bit 31	not used

Return Code

```

NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered

```

WD_AI_ContVScale

Converts the values of an array of acquired binary data from a continuous A/D conversion call to actual input voltages. The acquired binary data in the reading array might include the channel information (refer to continuous functions such as `WD_AI_ContReadChannel` or `WD_AI_ContScanChannels` for detailed data format). However, the calculated voltage values in the returned voltage array is not included the channel message.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_ContVScale (U16 CardNumber, U16  
AdRange, void *readingArray, F64 *voltageAr-  
ray, I32 count)
```

Visual Basic

```
WD_AI_ContVScale (ByVal CardNumber As Inte-  
ger, ByVal AdRange As Integer, readingArray As  
Integer, voltageArray As Double, ByVal count  
As Long) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

AdRange:

The analog input range the continuous specified channel is setting. Please refer to Section A.2 AI Range Codes for the valid range values.

readingArray:

Acquired continuous analog input data array.

voltageArray:

Computed voltages array returned.

count:

Amount of data to be converted.

Return Code

```
NoError  
ErrorInvalidCardNumber  
ErrorCardNotRegistered  
ErrorFuncNotSupport  
ErrorInvalidAdRange
```

WD_AI_ContVScaleEx

Converts the values of an array of binary data of specified width from a continuous A/D conversion call to actual input voltages.

Supported Cards

9852

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_ContVScaleEx (U16 CardNumber, U16
AdRange, U16 width, void *readingArray, F64*
voltageArray, I32 count)
```

Visual Basic

```
WD_AI_ContVScaleEx (ByVal CardNumber As Integer,
ByVal AdRange As Integer, ByVal width As Integer,
readingArray As Integer, voltageArray As Double,
ByVal count As Long) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

AdRange:

The analog input range the continuous specified channel is setting. Please refer to Section A.2 AI Range Codes for the valid range values.

width:

Width of input data.

readingArray:

Acquired continuous analog input data array.

voltageArray:

Computed voltages array returned.

count:

Amount of data to be converted.

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
```

ErrorInvalidAdRange

WD_AI_ConvertCheck

determines whether data conversion is completed.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_ConvertCheck (U16 CardNumber, BOOL-  
EAN *Stopped)
```

Visual Basic

```
WD_AI_ConvertCheck (ByVal CardNumber As Inte-  
ger, Stopped As Byte) As Integer
```

Parameters

CardNumber:

ID of the card performing asynchronous operation.

Stopped:

Indicates whether data conversion is completed. If Stopped = TRUE, data conversion has stopped. If Stopped = FALSE, data conversion is not yet completed. Constants TRUE and FALSE are defined in WD-DASK.h.

Return Code

```
NoError  
ErrorInvalidCardNumber  
ErrorCardNotRegistered  
ErrorFuncNotSupport
```

WD_AI_DMA_Transfer

Transfers the acquired data from the onboard SDRAM or DDR2 to the specified buffer. This function is only valid for two steps of data acquisition. If the buffer has a length smaller than the total data count, WD-DASK calls WD_AI_DMA_TransferBySize to fetch the data stored in the

onboard SDRAM or DDR2. Refer to `WD_AI_DMA_TransferBySize` for details.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++ and Borland C++

```
I16 WD_AI_DMA_Transfer (U16 CardNumber, U16
BufId)
```

Visual Basic

```
WD_AI_DMA_Transfer (ByVal CardNumber As Integer,
ByVal BufId As Integer) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

BufId:

The buffer ID (returned from function `WD_AI_ContBufferSetup`) of the buffer that will store the data from the SDRAM. If the BufId buffer has a smaller length than the total data count, you have to call this function repeatedly to fetch the data stored in the onboard SDRAM or DDR2. Refer to **Appendix A.3: AI Data Format** for the Build data format.

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
```

WD_AI_DMA_TransferBySize

Starts the transfer of acquired data from the onboard SDRAM or DDR2 to the specified buffer. This function is valid only for two steps of data acquisition. Since the amount of memory may be locked depending on the system, the driver may not lock memory as you specified.

WD-DASK allows the applications to break large-sized data transfer into several smaller data retrievals. The code segment for several smaller size of DMA transfer illustrated below:

```
WD_AI_Set_Mode (card, DAQSTEPPED, 1); //two steps
    data acquisition mode enabled

WD_AI_ContBufferSetup (card, ai_buf, locksize,
    &Id); //a small size of buffer locked

WD_AI_ContReadChannel (card, 0, Id, readcnt, 1,
    1, ASYNCH_OP);

do{
WD_AI_ConvertCheck(card, &fStop);
// check if DAQ conversion is completed
} while (!fStop);

//DAQ conversion stopped

do {
//smaller size of data transfer
WD_AI_DMA_TransferBySize (card, 10,
Id, locksize, &numRead, &remains, &complete);
} while(!complete); //check if all the data
//are read out

WD_AI_AsyncClear(card, &startPos, &count);
.....
```

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++ and Borland C++

```
I16 WD_AI_DMA_TransferBySize (U16 CardNumber,
F32 timeLimit, U16 BufId, U32 ReadCount, U32
*numRead, U32 *dataNotTransferred, U8 *com-
plete);
```

Visual Basic

```
WD_AI_DMA_TransferBySize (ByVal CardNumber As Integer, ByVal timeLimit As Single, ByVal BufId As Integer, ByVal ReadCount As Long, numRead As Long, dataNotTransferred As Long, complete As Byte) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

timeLimit:

Time limit (in seconds) for the DMA transfer operation. The default value is 1 second.

BufId:

The buffer ID (returned from function `WD_AI_ContBufferSetup`) of the buffer that will store the data from the SDRAM. If the `BufId` buffer has a smaller length than the total data count, you have to call this function repeatedly to fetch the data stored in the onboard SDRAM or DDR2. Refer to **Appendix A.3: AI Data Format** for the `BufId` data format.

ReadCount:

The count of data for transfer.

numRead:

The returned actual count of data transferred.

dataNotTransferred:

The returned count of data remaining in the onboard SDRAM or DDR2 (not read out).

complete

0	Some data remains in the onboard SDRAM or DDR2.
1	No data remains or all data in the onboard SDRAM or DDR2 has been read out.

Return Code

NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport

WD_AI_EventCallback

Controls and notifies the user's application when a specified DAQ event occurs. The notification is performed through a user-specified callback function.

In Linux, the event message has to be guidely removed by setting **mode** to 0. In Windows, the event message is removed automatically after calling `WD_AI_Async_Clear`. The event message may also be guidely removed by setting **mode** to 0.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++ and Borland C++

```
I16 WD_AI_EventCallback (U16 CardNumber, I16  
mode, I16 EventType, U32 callbackAddr)
```

Linux C++

```
I16 WD_AI_EventCallback (U16 CardNumber, I16  
mode, I16 EventType, void (*callback-  
Addr)(int))
```

Visual Basic

```
WD_AI_EventCallback (ByVal CardNumber As Inte-  
ger, ByVal mode As Integer, ByVal EventType As  
Integer, ByVal callbackAddr As Long) As Inte-  
ger
```

Parameters

CardNumber:

ID of the card performing the operation.

mode:

Adds or removes the event message. Valid values are:

0	Remove
1	Add

EventType:

Event criteria. Valid values are:

DAQEnd	Notifies that the asynchronous analog input operation has been completed.
DBEvent	Notifies that the next buffer of data in circular buffer is ready for transfer.
TrigEvent	Notifies that the buffer of data associated the next trigger is ready for transfer

callbackAddr:

Address of the user callback function. WD-DASK calls this function when the specified event occurs. If you want to remove the event message, set callbackAddr to 0.

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
```

WD_AI_InitialMemoryAllocated

Returns the available memory size for analog input in the device driver for argument MemSize. The continuous analog input transfer size may not exceed this size.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_InitialMemoryAllocated (U16 CardNumber, U32 *MemSize)
```

Visual Basic

```
WD_AI_InitialMemoryAllocated (ByVal CardNum-  
ber As Integer, MemSize As Long) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

MemSize:

Available memory size for continuous AI in the card's device driver. The unit is KB (1024 bytes).

Return Code

```
NoError  
ErrorInvalidCardNumber  
ErrorCardNotRegistered
```

WD_AI_Set_Mode

Configures the advanced mode of continuous AI operation. The advanced modes involves Data acquisition and storage to the SDRAM or DDR2, and transfer of data from the SDRAM or DDR2 to the system memory. After data is acquired, use `WD_AI_DMA_Transfer` or `WD_AI_DMA_TransferBySize` to fetch the data stored in the onboard SDRAM or DDR2, followed by re-start of continuous AI in kernel with single buffer or dual-buffers definitely or indefinitely.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_Set_Mode (U16 CardNumber, U16 modeC-  
trl, U16 Iter)
```

Visual Basic

```
WD_AI_Set_Mode (ByVal CardNumber As Integer,  
ByVal modeCtrl As Integer, ByVal Iter As Inte-  
ger) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

modeCtrl:

Advanced A/D mode. Valid values are:

DAQSTEPPED	Data is first acquired and stored to the SDRAM/DDR2, and then transferred to the system memory (NOT supported in PCIe-9842)
RestartEn	Restart continuous AI in kernel is enabled (NOT supported in PXIe-PCIe-9842/9814 and PXIe-9848 or PCIe/PXIe-9852)
DualBufEn	Dual-buffer mode for restart of continuous AI in kernel is enabled (NOT supported in PXIe-PCIe-9842/9814 and PXIe-9848 or PCIe/PXIe-9852)
ManualSoftTrg	Uses WD_SoftTriggerGen to generate the first software trigger signal
AI_AVE	AI average mode is enabled (PCIe/PXIe-9852 only)
AI_AVE_32	AI average mode with 32-bit data is enabled (PCIe/PXIe-9852 only)

When two constants are used to form the mode Ctrl argument, the constants are combined with the bitwise-OR operator(`|`).

Iter:

Iterations of data acquisition. If the value is zero, data acquisition is restarted infinitely.

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
```

WD_AI_SetTimeout

Informs WD-DASK library of the time limit for synchronous analog input operation to ensure that the function can return in a finite time interval.

Cards Support

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++ and Borland C++

```

I16 WD_AI_SetTimeout (U16 CardNumber, U32
msec)
  
```

Visual Basic

```

WD_AI_SetTimeout (ByVal CardNumber As Integer,
ByVal msec As Long) As Integer
  
```

Parameters

CardNumber:

ID of the card to perform this operation.

msec:

Time limit for analog input operation in unit of millisecond.

0	Wait indefinitely.
1~2147483648	The maximum time wait for synchronous AI operation.

Return Code

```

NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
  
```

WD_AI_Trig_Config

Informs the WD-DASK library of the trigger source, trigger mode, and trigger properties of the WD-DASK device with card ID CardNumber. After the WD_Register_Card function is called, the device is configured with default:

A/D trigger mode	WD_AI_TRGMOD_POST
A/D trigger source	WD_AI_TRGSRC_SOFT
A/D trigger polarity	WD_AI_TrgNegative

To use the device with the default settings, it is not necessary to call this function again to set the configuration. Otherwise, this function must be called before calling any function to perform continuous analog input.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_Trig_Config (U16 CardNumber, U16
TrigMode, U16 TrigSrc, U16 TrigPol, U16 AnaTrigchan, F64 AnaTriglevel, U32 PostTrigScans,
U32 PreTrigScans, U32 TrigDelayTicks, U32
TrgCnt)
```

Visual Basic

```
WD_AI_Trig_Config (ByVal CardNumber As Integer, ByVal TrigMode As Integer, ByVal TrigSrc
As Integer, ByVal TrigPol As Integer, ByVal AnaTrigChan As Integer, ByVal AnaTriglevel As
Double, ByVal PostTrigScans As Long, ByVal PreTrigScans As Long, ByVal TrigDelayTicks As
Long, ByVal TrgCnt As Long) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

TrigMode:

A/D trigger control setting.

WD_AI_TRGMOD_POST	Post Trigger Mode (default)
WD_AI_TRGMOD_DELAY	Delay Trigger Mode
WD_AI_TRGMOD_PRE	Pre-Trigger Mode (not supported by PCIe-9842)
WD_AI_TRGMOD_MIDL	Middle-Trigger Mode (not supported by PCIe-9842)

TrigSrc:

A/D trigger source setting.

WD_AI_TRGSRC_SOFT	Software (Default)
WD_AI_TRGSRC_ANA	From analog trigger pin (not supported by PCIe-9842)
WD_AI_TRGSRC_ExtD	From external digital trigger pin
WD_AI_TRSRC_SSI_1	Trigger event signal is from an SSI source
WD_AI_TRSRC_SSI_2	Both acquisition start and clocked trigger event signal are from SSI (not supported by PCIe-9842, PXIe-9848, PCIe-9814, and PCIe/PXIe-9852).
WD_AI_TRSRC_PXIStart	Trigger event signals are from PXI_START pin. (Valid only for PXI modules not installed in slot 2).
WD_AI_TRSRC_PXIeStar	Trigger event signals are from PXIe_START pin

TrigPol:

A/D trigger signal polarity setting.

WD_AI_TrgPositive	Trigger positive edge active (Default)
WD_AI_TrgNegative	Trigger negative edge active

AnaTrigchan:

Analog trigger channel setting.

CH0ATRIG	AI channel 0
CH1ATRIG	AI channel 1
CH2ATRIG	AI channel 2
CH3ATRIG	AI channel 3
CH4ATRIG	AI channel 4
CH5ATRIG	AI channel 5
CH6ATRIG	AI channel 6
CH7ATRIG	AI channel 7

TrigLevel:

Analog trigger level setting, or digital trigger threshold. The valid range depends on the AI range:

AD_B_5_V	-5V ~ +5V
AD_B_1_V	-1V ~ +1V
AD_B_10_V	-10V ~ +10V
AD_B_2_V	-2V ~ +2V
AD_B_0_2_V	-0.2V ~ +0.2V

For Digital Triggering, the valid digital trigger threshold is 0 to 3.3V, where 0.0 indicates the threshold value is maintained at the previous setting. Initial setting for the value is 1.67V.

PostTrigScans:

This argument is valid only for middle trigger. For middle trigger, postTrigScans indicates the number of data that will be accessed after a specific trigger event.

PreTrigScans:

This argument is valid only for middle trigger and pre-trigger. The number of scans of data WD-DASK collects before searching for the first trigger. Setting preTrigScans to 0 causes the WD-DASK to search for the first trigger as soon as the DAQ process begins. The valid value is 1 to 2147483647. for PCIe/PXle-9852, if the trigger source is SSI and the collected data has reached the preTrigscans, a pre-Data-Ready signal informs the SSI master device that it (SSI slave) can receive the signal. See “WD_Route_Signal” on page 94.

TrigDelayTicks:

Number of timer ticks that will be ignored after a specific trigger event.

TrgCnt:

Supported trigger times in an acquisition. This argument is valid only for delay trigger and post trigger modes. The valid

range is 1 to 2147483647. for PCIe/PXle-9852, if AI Average mode is enabled, valid range is 1 to 65535.



NOTE:

For pre-trigger and middle trigger modes of AI, the ReadScans parameter of the following continuous AI functions must equal postTrigScans+preTrigScans:

- ▶ WD_AI_ContReadChannel
- ▶ WD_AI_ContReadChannelToFile
- ▶ WD_AI_ContReadMultiChannels
- ▶ WD_AI_ContReadMultiChannelsToFile
- ▶ WD_AI_ContScanChannels
- ▶ WD_AI_ContScanChannelsToFile

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
```

WD_AI_VoltScale

Converts raw AD data to actual input voltage.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_VoltScale (U16 CardNumber, U16
AdRange, I16 reading, F64 *voltage)
```

Visual Basic

```
WD_AI_VoltScale (ByVal CardNumber As Integer,
ByVal AdRange As Integer, ByVal reading As
Integer, voltage As Double) As Integer
```

Parameters

CardNumber:

ID of the card to perform this operation.

AdRange:

The analog input range the specified channel is setting. Please refer to Section A.2 AI Range Codes for the valid range values.

reading:

Raw data of the AD conversion.

voltage:

Computed voltage value.

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
ErrorInvalidAdRange
```

WD_AI_VoltScale32

Converts raw AD data to actual input voltage.

Supported Cards

9852

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AI_VoltScale32 (U16 CardNumber, U16
AdRange, I32 reading, F64 *voltage)
```

Visual Basic

```
WD_AI_VoltScale32 (ByVal CardNumber As Integer,
ByVal AdRange As Integer, ByVal reading
As Long, voltage As Double) As Integer
```

Parameters

CardNumber:

ID of the card to perform this operation.

AdRange:

The analog input range the specified channel is setting. Please refer to Section A.2 AI Range Codes for the valid range values.

reading:

Raw data of the AD conversion.

voltage:

Computed voltage value.

Return Code

```
NoError  
ErrorInvalidCardNumber  
ErrorCardNotRegistered  
ErrorFuncNotSupport  
ErrorInvalidAdRange
```

WD_OutTrig_Config

Reports properties of the TRIG_OUT signals for the WD-DASK device with card ID CardNumber to the WD-DASK library.

Supported Cards

9842, 9852

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
U16 WD_OutTrig_Config (U16 wCardNumber, U16  
trig_Ch, U16 trig_conf, U16 trig_width)
```

Visual Basic

```
WD_OutTrig_Config (ByVal CardNumber As Integer,  
ByVal trigout_ch As Integer, ByVal  
trg_conf As Integer, ByVal trig_width As Integer)  
As Integer
```

Parameters

CardNumber:

ID of the card performing the operation

TrigOut_ch:

The TRIG_OUT channel number to be configured, with:

valid value of PCIe-9842/9852: 0

valid value of PXIe-9842/9852: AITRIGOUT_CH0: 0

AITRIGOUT_PXI : 2

AITRIGOUT_PXI_TRIG_n (n : 0 - 7): 2 through 9

Trig_Conf:

Ignored

trig_width:

Output pulse width, with valid values of
 WD_OutTrgPWidth_50ns, WD_OutTrgPWidth_100ns,
 WD_OutTrgPWidth_150ns, WD_OutTrgPWidth_200ns,
 WD_OutTrgPWidth_500ns, WD_OutTrgPWidth_1us,
 WD_OutTrgPWidth_2us, WD_OutTrgPWidth_5us, and
 WD_OutTrgPWidth_10us

Return Code

NoError
 ErrorInvalidCardNumber
 ErrorCardNotRegistered
 ErrorFuncNotSupport

2.3.2 Digital Input/ Output

WD_DI_ReadLine

Reads the digital logic state of the specified digital line in the specified port.

Supported Cards

9842

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_DI_ReadLine (U16 CardNumber, U16 Port,
U16 Line, U16 *State)
```

Visual Basic

```
WD_DI_ReadLine (ByVal CardNumber As Integer,
ByVal Port As Integer, ByVal Line As Integer,
Value As Integer) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation

Port:

Digital input port number

Line:

Digital line to be read. The valid value is 0 through 7

State:

Returns digital logic state, 0 or 1, of the specified line.

Return Code

NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport

WD_DI_ReadPort

Reads digital data from the specified digital input port.

Supported Cards

9842

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_DI_ReadPort (U16 CardNumber, U16 Port,  
U32 *Value)
```

Visual Basic

```
WD_DI_ReadPort (ByVal CardNumber As Integer,  
ByVal Port As Integer, Value As Long) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation

Port:

Digital input port number

ValueReturns:

The digital data read from the specified port. The returned value is a 8-bit data

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
```

WD_DIO_LineConfig

Informs WD-DASK library of the DI signal source or the direction (Input or Output) setting of the selected lines.

Supported Cards

9842, 9852

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_DIO_LineConfig (U16 CardNumber, U16
Port, U32 Line, U16 configValue)
```

Visual Basic

```
WD_DIO_LineConfig (ByVal CardNumber As Integer,
ByVal Port As Integer, ByVal line As Long,
ByVal configValue As Integer) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation

Port:

Digital input port number

Line:

The line selected

ConfigValue:

For 9842, ConfigValue sets the direction of the specified line with valid values of INPUT_LINE and OUTPUT_LINE.

For 9852, ConfigValue sets where the SDI signal comes from. Each bit of the value of ConfigValue is corresponding to the related SSI line and (only bit can be 1). For SDI_0, bits 0-3 are mapped to SSI line 0-3 and for SDI_1 the bit 0-3 are mapped to SSI line 4-7. The '1' value of the bit value specifies the mapping SSI line where the SDI signal comes from.

Return Code

```
NoError  
ErrorInvalidCardNumber  
ErrorCardNotRegistered  
ErrorFuncNotSupport  
ErrorConfigFailed
```

WD_DIO_LinesConfig

Informs WD-DASK library of the entire lines of the port selected and the direction (Input or output) setting of the entire lines of the selected port (see reference documentation for modification).

Supported Cards

9842, 9852

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_DIO_LinesConfig (U16 CardNumber, U16  
Port, U16 mode, U32 Linesdirmap)
```

Visual Basic

```
WD_DIO_LinesConfig (ByVal CardNumber As Integer,  
ByVal Port As Integer, ByVal mode As Integer,  
ByVal wLinesdirmap As Long) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation

Port:

Digital input port number

Mode:

SDI mode selection, with valid values of SDI_En: 0 (default), and SDI_Dis:1

Linesdirmap:

For PXIe-9842, each bit of the value of Linesdirmap controls one line of the port selected. The '1' value of the bit value sets the corresponding line to output, and the '0' value of the bit value sets the corresponding line to input, with valid values of 0 through 0xff.

For PXIe-9852, each bit of the value of Linesdirmap corresponds to the SSI lines from where the SDI signals come. The '1' value of the bit value enables the corresponding SSI line. Bit0-3 to control SDI_0 (only bit can be 1) and bit4-7 controls SDI_1 (only bit can be 1).

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
ErrorConfigFailed
```

WD_DIO_PortConfig

Informs WD-DASK library of the port selected and the direction (Input or output) setting of the selected port (see reference documentation for modification).

Supported Cards

9842, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_DIO_PortConfig (U16 CardNumber, U16
Port, U16 mode, U16 Direction)
```

Visual Basic

```
WD_DIO_PortConfig (ByVal CardNumber As Integer, ByVal Port As Integer, ByVal mode As Integer, ByVal Direction As Integer) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation

Port:

Digital input port number

ModeSDI:

Mode selection, with valid values of: SDI_En: 0 (default), and SDI_Dis:1

Direction:

For PXIe-9842, the port direction of the PI/O port, with valid values of INPUT_PORT and OUTPUT_PORT.

For PXIe-9852, each bit of the value of Direction corresponding to the SSI lines from where the SDI signals come. The '1' value of the bit value enables the corresponding SSI line. Bit0-3 controls SDI_0 (only bit can be 1) and bit4-7 controls SDI_1 (only bit can be 1).

Return Code

```
NoError  
ErrorInvalidCardNumber  
ErrorCardNotRegistered  
ErrorFuncNotSupport  
ErrorConfigFailed
```

WD_DO_ReadLine

Reads back the digital logic state of the specified digital output line in the specified port

Supported Cards

9842

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_DO_ReadLine (U16 CardNumber, U16 Port,
U16 Line, U16 *State)
```

Visual Basic

```
WD_DO_ReadLine (ByVal CardNumber As Integer,
ByVal Port As Integer, ByVal Line As Integer,
Value As Integer) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation

Port:

Digital output port number

Line:

The digital line to be read. The valid value is 0 through 7

State:

Returns the digital logic state, 0 or 1, of the specified line

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
```

WD_DO_ReadPort

Reads back the output digital data from the specified digital output port.

Supported Cards

9842

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_DO_ReadPort (U16 CardNumber, U16 Port,
U32 *Value)
```

Visual Basic

```
WD_DO_ReadPort (ByVal CardNumber As Integer,  
ByVal Port As Integer, Value As Long) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation

Port:

Digital input port number

Value:

Returns digital data read from the specified port. The returned value is a 8-bit data

Return Code

```
NoError  
ErrorInvalidCardNumber  
ErrorCardNotRegistered  
ErrorFuncNotSupport  
ErrorTransferCountTooLarge  
ErrorContIoNotAllowed
```

2.3.3 Configuration

WD_Register_Card

Initializes the hardware and software states of a WD-DASK data acquisition card, then returns a numeric card ID that corresponds to the initialized card. This function must be called before any other WD-DASK library functions. The function initializes the card and variables internal to the WD-DASK library. Because WD-DASK devices come in plug-and-play design, the base address (pass-through address) and IRQ level are automatically assigned by the system BIOS.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_Register_Card (U16 CardType, U16
card_num)
```

Visual Basic

```
WD_Register_Card (ByVal CardType As Integer,
ByVal card_num As Integer) As Integer
```

Parameter

CardType:

The type of card to be initialized. ADLINK periodically upgrades WD-DASK to support new digitizer data acquisition cards. Refer to card's Release Notes to know if WD-DASK supports that card. The following is a constant defined in WD-DASK.h.

PCI_9820 (for PCI-9820/PXI_9820)
PXI devices:
PXI_9816D
PXI_9826D
PXI_9846D
PXI_9846DW (wide bandwidth)
PXI_9816H
PXI_9826H
PXI_9846H
PXI_9846HW (wide bandwidth)
PXI_9816V
PXI_9826V
PXI_9846V
PXI_9846VW (wide bandwidth)
PCI devices:
PCI_9816D
PCI_9826D
PCI_9846D
PCI_9846DW (wide bandwidth)

PCI_9816H
PCI_9826H
PCI_9846H
PCI_9846HW (wide bandwidth)
PCI_9816V
PCI_9826V
PCI_9846V
PCI_9846VW (wide bandwidth)
PCIe devices:
PCIe_9814
PCIe_9816D
PCIe_9826D
PCIe_9846D
PCIe_9846DW (wide bandwidth)
PCIe_9816H
PCIe_9826H
PCIe_9846H
PCIe_9846HW (wide bandwidth)
PCIe_9816V
PCIe_9826V
PCIe_9846V
PCIe_9846VW (wide bandwidth)
PCIe_9852
PCIe_9842 (for PCIe_9842/PXIe-9842)
PXIe devices:
PXIe_9848
PXIe_9852

card_num:

Sequence number of a card with the same card type as defined in CardType and also installed in the PCI/PXI slot. The card sequence number setting is based on the PCI slot sequence in the motherboard. The first card (installed in the first slot) is assigned with card_num=0. For example, if there are two PCI-9820 cards installed in your computer, the

PCI-9820 card installed in the first slot is registered with `card_num=0`, while the next one is registered with `card_num=1`.

Return Code

This function returns a numeric card ID for the initialized cards. ID range is between 0 and 31. If any error occurred, the function returns a negative error code. Below are the possible error codes:

```
ErrorTooManyCardRegistered
ErrorUnknownCardTypes
ErrorOpenDriverFailed
ErrorOpenEventFailed
```

WD_Register_Card_By_PXISlot_GA

Initializes the hardware and software states of a WD_DASK device with the geographic address of the PXI/PXIe slot into which it is plugged and returns a numeric card ID corresponding to the card initialized. `WD_Register_Card` or `WD_Register_Card_By_PXISlot_GA` (only for PXI/PXIe module) must be called before any other WD-DASK library functions can be called for that card. The function initializes the card and variables internal to WD-DASK library. To meet plug-and-play requirements, the base address (pass-through address) and IRQ level are assigned directly by system BIOS.

Supported Cards

PXI or PXIe modules

Syntax

Microsoft C/C++, Linux C/C++ and Borland C++

```
I16 WD_Register_Card_By_PXISlot_GA (U16 Card-
Type, U16 geo_addr)
```

Visual Basic

```
WD_Register_Card_By_PXISlot_GA (ByVal card-
Type As Integer, ByVal geo_addr As Integer) As
Integer
```

Parameters

CardType:

Type of card to be initialized. Please refer to WD_Register_Card function.

geo_addr:

Geographic address of the card mounted in the PXI slot.

Return Code

Returns a numeric card id for the card initialized. The range of card id is between 0 and 31. If any error occurs, it will return a negative error code, with possible error codes:

```
ErrorTooManyCardRegistered  
ErrorOpenDriverFailed  
ErrorOpenEventFailed
```

WD_Release_Card

You may simultaneously register up to to 32 cards. This function tells the WD-DASK library that the registered card is not currently in used and may be released. This would make room for a new card to register. Also, at the end of every program, you need to use this function to release all registered cards.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_Release_Card (U16 CardNumber)
```

Visual Basic

```
WD_Release_Card (ByVal CardNumber As Integer)  
As Integer
```

Parameter

CardNumber:

ID of the card for release.

Return Code

NoError

WD_Buffer_Alloc

Allocate memory with the specified bytes. If the memory is not needed, using `WD_Buffer_free` to release the allocated memory.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
void* WD_Buffer_Alloc (U16 wCardNumber, U32
dwSize)
```

Visual Basic

```
WD_Buffer_Alloc (ByVal CardNumber As Short,
ByVal dwSize As Long) As Long
```

Parameters

CardNumber:

ID of the card performing the operation

Size:

Size (in bytes) to be allocated

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
ErrorTransferCountTooLarge
ErrorContIoNotAllowed
```

WD_Buffer_Free

Free the buffer allocated by `WD_Buffer_Alloc`.

Supported Cards:

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax:

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_Buffer_Free (U16 wCardNumber, void  
*buf)
```

Visual Basic

```
WD_Buffer_Free (ByVal CardNumber As Short,  
Buffer As Any) As Long
```

Parameters

CardNumber:

ID of the card performing the operation.

Buffer:

Starting address of the memory wishing to be deleted.

Return Code

```
NoError  
ErrorInvalidCardNumber  
ErrorCardNotRegistered  
ErrorFuncNotSupport  
ErrorTransferCountTooLarge  
ErrorContIoNotAllowed
```

2.3.4 Calibration

WD_AD_Auto_Calibration_ALL

Calibrates the WD-DASK device. When this function is called, the device goes into a self-calibration cycle. The function does not return until self-calibration is completed.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_AD_Auto_Calibration_ALL(U16 CardNumber)
```

Visual Basic

```
WD_AD_Auto_Calibration_ALL (ByVal CardNumber
As Integer) As Integer
```

Parameter

CardNumber:

ID of the card performing asynchronous operation

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
ErrorInvalidIoChannel
```

WD_EEPROM_CAL_Constant_Update

Saves the new calibration constants to the specified EEPROM bank.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_EEPROM_CAL_Constant_Update (U16 CardNum-
ber, U16 bank)
```

Visual Basic

```
WD_EEPROM_CAL_Constant_Update (ByVal wCardNum-
ber As Integer, ByVal bank As Integer) As Inte-
ger
```

Parameters

CardNumber:

ID of the card performing the operation.

bank:

Storage location on the EEPROM. The valid range for PCIe-9842 is 1 to 3, PXIe-9848 is 1 to 2, PCIe-9814 and PCIe/PXIe-9852 is 1, and for other modules 0 to 3.

Return Code

```
NoError  
ErrorInvalidCardNumber  
ErrorCardNotRegistered  
ErrorFuncNotSupport  
ErrorInvalidIoChannel
```

WD_Get_Default_Load_Area

Acquires the number of the bank in which the calibration data is loaded by default when WD_Register_Card is called.

Supported Cards

9842, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_Get_Default_Load_Area(U16 CardNumber)
```

Visual Basic

```
WD_Get_Default_Load_Area (ByVal CardNumber As  
Integer) As Integer
```

Parameter

CardNumber:

ID of the card performing the operation

Return Code

The default bank number

WD_Load_CAL_Data

Loads the calibration constants from the specified EEPROM bank.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_Load_CAL_Data (U16 CardNumber, U16
bank)
```

Visual Basic

```
WD_Load_CAL_Data (ByVal CardNumber As Integer,
ByVal bank As Integer) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

bank:

Storage bank on the EEPROM. The valid range is 0 to 3.

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
ErrorInvalidIoChannel
```

WD_Set_Default_Load_Area

Reports the number of the bank in which the calibration data is loaded by default when `WD_Register_Card` is called, to the WD-DASK library.

Supported Cards

9842, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_Set_Default_Load_Area (U16 CardNumber,
U16 bank)
```

Visual Basic

```
WD_Set_Default_Load_Area (ByVal CardNumber As
Integer, ByVal bank As Integer) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation

bank:

The storage location on EEPROM. The valid range of value for bank is 1 through 2, and default value is 1.

Return Code

```
NoError  
ErrorInvalidCardNumber  
ErrorCardNotRegistered  
ErrorFuncNotSupport
```

2.3.5 SSI

WD_Route_Signal

Routes an internal signal to the SSI line, PXI STAR line, or PXI trigger bus line, and/or enables clock sharing through the PXI trigger bus line or the PXI STAR line.

Supported Cards

9820, 9816, 9826, 9842, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_Route_Signal (U16 CardNumber, U16 sig-  
nal, U16 Line, U16 dir)
```

Visual Basic

```
WD_Route_Signal (ByVal CardNumber As Integer,  
ByVal signal As Integer, ByVal Line As Inte-  
ger, ByVal dir As Integer) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

signal:

Signal routed to the specified line. Valid signal codes are:

SSSI_TIME	Clock signal
-----------	--------------

SSI_TRIG_SRC1	Trigger event signal
SSI_TRIG_SRC2_S	Acquisition start trigger signal output
SSI_TRIG_SRC2_T	Clocked trigger signal output
SSI_PRE_DATA_RDY	preTrigger Data Ready signal for pre/middle trigger mode (ONLY available on PCIe/PXIe-9852)

For modules supporting SSI_PRE_DATA_RDY (e.g. PCIe/PXIe-9852) operated in pre-trigger or middle trigger mode, when the collected data reaches the preTrigscans, the device issues a Pre-Data-Ready signal to inform the SSI master that the SSI slave is ready to receive the trigger signal. The SSI slave routes the SSI_PRE_DATA_RDY signal to a specified line also set by the master to receive the SSI_PRE_DATA_RDY signal. When acquiring SSI_PRE_DATA_RDY signals from all SSI slaves, acquiring data to the preTrigscans and receiving the AI trigger signal, the SSI master routes the AI trigger signal through SSI cable or PXIe trigger line to the SSI slaves awaiting the trigger signal.

To configure the SSI slave to output SSI_PRE_DATA_RDY signal through SSI_LINE_0 and the SSI master to receive the SSI_PRE_DATA_RDY signal:

For SSI slave:

```
WD_Route_Signal (slave_card, SSI_PRE_DATA_RDY,
                 SSI_LINE_0, 1);
```

SSI master:

```
WD_Route_Signal (master_card, SSI_PRE_DATA_RDY,
                 SSI_LINE_0, 0);
```

Line:

Trigger line to drive. Valid lines for PXI/PXIe module include:

PXI_TRIG_n	PXI trigger bus lines (n is the line number with a value of 0 to 7)
PXI_START_TRIG	PXI STAR line

The following line is valid for both PXI and PCI modules:

TRG_IO	TRG_IO pin
--------	------------

The following line is valid for PCI/PCIe modules including:

SSI_LINE_n	SSI cable lines (n is the line number with a value of 0 to 7) for PCIe-9852, the valid lines for SSI_PRE_DATA_RDY are SSI_LINE_0, SSI_LINE_1, SSI_LINE_5 and SSI_LINE_6
------------	--

dir.

direction of the connection.

0	Receive signal from the connection line. This value is not valid for PCI modules.
1	Transmit signal to the connection line.



NOTE:

PXI-9816/26/46, PXIe-9842, and PXIe-9852 cannot be PXI STAR signal output source, but can accept PXI STAR signal input

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
ErrorNotStartTriggerModule
ErrorInvalidRouteLine
ErrorInvalidSignalCode
ErrorInvalidSignalDirection
```

WD_Signal_DisConn

Disconnects a device signal from the SSI line, PXI_START line, or the PXI trigger bus line.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_Signal_DisConn (U16 CardNumber, U16
    signal, U16 Line)
```

Visual Basic

```
WD_Signal_DisConn (ByVal CardNumber As Integer,
    ByVal signal As Integer, ByVal Line As Integer) As Integer
```

Parameters

CardNumber.

ID of the card performing the operation.

signal:

Signal routed to the specified line. Valid signal codes are:

SSSI_TIME	Clock signal
SSI_TRIG_SRC1	Trigger event signal
SSI_TRIG_SRC2_S	Acquisition start trigger signal output
SSI_TRIG_SRC2_T	Clocked trigger signal output
SSI_PRE_DATA_RDY	preTrigger Data Ready signal for pre/middle trigger mode (ONLY available on PCIe/PXle-9852)

For modules supporting SSI_PRE_DATA_RDY (e.g. PCIe/PXle-9852) operated in pre-trigger or middle trigger mode, when the collected data reaches the preTrigscans, the device issues a Pre-Data-Ready signal to inform the SSI master that the SSI slave is ready to receive the trigger signal. The SSI slave routes the SSI_PRE_DATA_RDY signal to a specified line also set by the master to receive the SSI_PRE_DATA_RDY signal. When acquiring SSI_PRE_DATA_RDY signals from all SSI slaves, acquiring data to the preTrigscans and receiving the AI trigger signal, the SSI master routes the AI trigger signal through SSI cable or PXle trigger line to the SSI slaves awaiting the trigger signal.

To configure the SSI slave to output SSI_PRE_DATA_RDY signal through SSI_LINE_0 and the SSI master to receive the SSI_PRE_DATA_RDY signal:

For SSI slave:

```
WD_Route_Signal (slave_card, SSI_PRE_DATA_RDY,
                  SSI_LINE_0, 1);
```

SSI master:

```
WD_Route_Signal (master_card, SSI_PRE_DATA_RDY,
                  SSI_LINE_0, 0);
```

Line:

Trigger line to drive. Valid lines for PXI/PXIe module include:

PXI_TRIG_n	PXI trigger bus lines (n is the line number with a value of 0 to 7)
PXI_START_TRIG	PXI STAR line

The following line is valid for both PXI and PCI modules:

TRG_IO	TRG_IO pin
--------	------------

The following line is valid for PCI/PCIe modules including:

SSI_LINE_n	SSI cable lines (n is the line number with a value of 0 to 7) for PCIe-9852, the valid lines for SSI_PRE_DATA_RDY are SSI_LINE_0, SSI_LINE_1, SSI_LINE_5 and SSI_LINE_6
------------	--

dir:

direction of the connection.

0	Receive signal from the connection line. This value is not valid for PCI modules.
1	Transmit signal to the connection line.

Return Code

```
NoError
ErrorInvalidCardNumber
```

```
ErrorCardNotRegistered
ErrorFuncNotSupport
ErrorFuncNotSupport
ErrorInvalidRouteLine
ErrorInvalidSignalCode
ErrorInvalidSignalDirection
```

WD_SSI_SourceClear

Disconnects all device signals from the SSI bus trigger, PXI_STAR line, or the PXI trigger bus lines.

Supported Cards

9820, 9816, 9826, 9842, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_SSI_SourceClear (U16 CardNumber)
```

Visual Basic

```
WD_SSI_SourceClear (ByVal CardNumber As Integer) As Integer
```

Parameter

CardNumber.

ID of the card performing the operation.

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
```

WD_SoftTriggerGen

Generates a software trigger signal.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_SoftTriggerGen (U16 CardNumber, U8 op)
```

Visual Basic

```
WD_SoftTriggerGen (ByVal CardNumber As Integer, ByVal op As Byte) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

op:

The operation that awaits the software trigger signal, where Valid op code is

```
SOFTTRIG_AI
```

which starts or stops trigger for analog input operation.

Return Code

```
NoError  
ErrorInvalidCardNumber  
ErrorCardNotRegistered  
ErrorFuncNotSupport  
InvalidCounter
```

WD_SSI_SourceConn

Connects a device to the specified SSI bus trigger line. This function is valid only for PCI/PCIe modules.

Supported Cards

PCI-9820, PCI-9816, PCI-9826, PCI-9846, PCIe-9852, PCIe-9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_SSI_SourceConn (U16 CardNumber, U16 sigCode)
```

Visual Basic

```
WD_SSI_SourceConn (ByVal CardNumber As Integer, ByVal sigCode As Integer) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

sigCode:

Specified SSI signal code number of the device signal to be disconnected from the SSI bus trigger line. Valid signal codes are:

SSI_TIME	SSI_TIMEBASE output
SSI_TRIG_SSI1	Trigger event signal output
SSI_TRIG_SSI2	Both acquisition start and clocked trigger signal output

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
```

WD_SSI_SourceDisConn

Disconnects a device signal from the specified SSI bus trigger line. This function is valid only for PCI modules.

Supported Cards

PCI-9820, PCI-9816, PCI-9826, PCI-9846, PCIe-9852, PCIe-9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_SSI_SourceDisConn (U16 CardNumber, U16
sigCode)
```

Visual Basic

```
WD_SSI_SourceDisConn (ByVal CardNumber As
Integer, ByVal sigCode As Integer) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

sigCode:

Specified SSI signal code number of the device signal to be disconnected from the SSI bus trigger line. Valid signal codes are:

SSI_TIME	SSI_TIMEBASE output
SSI_TRIG_SSI1	Trigger event signal output
SSI_TRIG_SSI2	Both acquisition start and clocked trigger signal output

Return Code

NoError
 ErrorInvalidCardNumber
 ErrorCardNotRegistered
 ErrorFuncNotSupport

2.3.6 Other

WD_Device_Scan

Scans PCI(e)/PXI(e) buses and registers all active installed WD-DASK devices.

Supported Cards

All WD-DASK modules

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_Device_Scan(WD_DEVICE* AvailModules,
U16 ncount, U16* retcnt)
```

Visual Basic

```
WD_Device_Scan (AvailModules As WD_DEVICE,
ByVal ncount As Integer, retcnt As Integer) As
Integer
```

Parameters

AvailModules:

User provides WD_DASK array storing the information of active installed WD-DASK modules. The WD_DASK structure is defined in wd-dask/wddask64.h as:

```
typedef struct
{
    U16 wModuleType;
    U16 wCardID;
    U16 geo_addr;
    U16 Reserved;
    char dispname[64];
} WD_DEVICE, *PWD_DEVICE

ncount:
    Available device count of AvailModules

retcnt:
    Points to a variable receiving the number of active installed
    wd-dask modules
```

Return Code

```
NoError
ErrorOpenDriverFailed
```

WD_Get_SDRAMSize

Get the size of onboard SDRAM in unit of MBytes.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_Get_SDRAMSize (U16 CardNumber, U32
*sdramsize)
```

Visual Basic

```
WD_Get_SDRAMSize (ByVal CardNumber As Integer,
sdramsize As Long) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

sdramsize:

Size of the onboard SDRAM in unit MBytes.

Return Code

```
NoError  
ErrorInvalidCardNumber  
ErrorCardNotRegistered  
ErrorFuncNotSupport  
ErrorInvalidIoChannel
```

WD_GetSerialNumber

Returns the serial number (S/N).of the module, where the returned string matches the serial number listed on the device label.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_GetSerialNumber(U16 wCardNumber, U8  
*SerialString, U8 numberOfElements, U8* actu-  
alread)
```

Visual Basic

```
WD_GetSerialNumber (ByVal CardNumber As Inte-  
ger, SerialString As Byte, ByVal numberOfEle-  
ments As Byte, actualread As byte) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

SerialString:

Pointer to a buffer receiving the serial number string. The length of the serial for wd-dask modules is 10.

numberOfElements:

Length of the SerialString buffer, in characters, with minimum buffer size 10.

actualread:

The actual size of the serial number string.

Return Code

```
NoError
ErrorInvalidCardNumber
ErrorCardNotRegistered
ErrorFuncNotSupport
```

WD_GetPXIGeographAddr

Obtains the number of the physical slot where the specified PXI module is installed, or the board ID setting for a device with a Board ID DIP switch on the board.

Supported Cards

9820, 9816, 9826, 9842, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_GetPXIGeographAddr (U16 CardNumber, U8*
geo_addr)
```

Visual Basic

```
WD_GetPXIGeographAddr (ByVal CardNumber As
Integer, geo_addr As Byte) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

geo_addr:

Number of the physical slot where the specified module is installed.

Return Code

```
NoError
```

```
ErrorInvalidCardNumber  
ErrorCardNotRegistered  
ErrorFuncNotSupport  
ErrorInvalidIoChannel
```

WD_GetFPGAVersion

Returns the onboard firmware version.

Supported Cards

All WD-DASK modules

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_GetFPGAVersion (U16 wCardNumber, U32*  
version)
```

Visual Basic

```
WD_GetFPGAVersion (ByVal CardNumber As Inte-  
ger, version As Long) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

Version:

Returns the firmware version

Return Code

```
NoError  
ErrorInvalidCardNumber  
ErrorCardNotRegistered  
ErrorFuncNotSupport
```

WD_MeasureStats

Returns the measurement stat from the specified channels from a previous waveform measurement.

Supported Cards

9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_MeasureStats (U16 CardNumber, U16 wNum-
Chans, U16 *chans, U16 item, void* valBuf)
```

Visual Basic

```
WD_MeasureStats (ByVal CardNumber As Integer,
ByVal numChans As Integer, ByVal reading As
Long, voltage As Double) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

NumChans:

Number of analog input channels in the array Chans, with Valid values 1 to 4.

Chans:

Array of analog input channel numbers, between 0 and 3.

item:

Returns measured item, with Valid items VAL_MAX and VAL_MIN.

WD_SoftTriggerGen

WD_SoftTriggerGen

Generates a software trigger signal.

Supported Cards

9820, 9816, 9826, 9846, 9848, 9852, 9814

Syntax

Microsoft C/C++, Linux C/C++, and Borland C++

```
I16 WD_SoftTriggerGen (U16 CardNumber, U8 op)
```

Visual Basic

```
WD_SoftTriggerGen (ByVal CardNumber As Inte-
ger, ByVal op As Byte) As Integer
```

Parameters

CardNumber:

ID of the card performing the operation.

op:

The operation that awaits the software trigger signal. Valid op code as shown

SOFTTRIG_AI	Starts or stops trigger for analog input operation
-------------	--

Return Code

NoError

ErrorInvalidCardNumber

ErrorCardNotRegistered

ErrorFuncNotSupport

InvalidCounter

Appendix A Codes and Formats

A.1 Status Codes

This appendix lists the status codes returned by WD-DASK, including the name and description.

Each WD-DASK function returns a status code that indicates whether the function was performed successfully. When a WD-DASK function returns a negative number, it means that an error occurred while executing the function.

Status Code	Name	
0	NoError	No error occurred.
-1	ErrorUnknownCardType	The CardType argument is not valid.
-2	ErrorInvalidCardNumber	The CardNumber argument is out of range (larger than 31).
-3	ErrorTooManyCardRegistered	32 cards have been registered.
-4	ErrorCardNotRegistered	No card registered with id CardNumber.
-5	ErrorFuncNotSupport	The function called is not supported.
-6	ErrorInvalidIoChannel	The specified Channel or Port argument is out of range.
-7	ErrorInvalidAdRange	The specified analog input range is invalid.
-8	ErrorContIoNotAllowed	The specified continuous IO operation is not supported.
-9	ErrorDiffRangeNotSupport	All analog input ranges must be the same for multi-channel analog input.
-10	ErrorLastChannelNotZero	The channels for multi-channel analog input must end with or start from zero.
-11	ErrorChannelNotDescending	The channels for multi-channel analog input must be contiguous and in descending order.

Status Code	Name	
-12	ErrorChannelNotAscending	The channels for multi-channel analog input must be contiguous and in ascending order.
-13	ErrorOpenDriverFailed	Failed to open the device driver.
-14	ErrorOpenEventFailed	Open event failed in the device driver.
-15	ErrorTransferCountTooLarge	The size of transfer is larger than the size of initially allocated memory in the driver.
-16	ErrorNotDoubleBufferMode	Double-buffer mode is disabled.
-17	ErrorInvalidSampleRate	The specified sampling rate is out of range.
-18	ErrorInvalidCounterMode	Value of the Mode argument is invalid.
-19	ErrorInvalidCounter	Value of the Ctr argument is out of range.
-20	ErrorInvalidCounterState	Value of the State argument is out of range.
-21	ErrorInvalidBinBcdParam	Value of the BinBcd argument is invalid.
-22	ErrorBadCardType	Value of CardType argument is invalid.
-23	ErrorInvalidDaRefVoltage	Value of DA reference voltage argument is invalid.
-24	ErrorAdTimeOut	AD operation timed-out.
-25	ErrorNoAsyncAI	Continuous Analog Input is not set in asynchronous mode
-26	ErrorNoAsyncAO	Continuous Analog Output is not set in asynchronous mode.
-27	ErrorNoAsyncDI	Continuous Digital Input is not set in asynchronous mode.
-28	ErrorNoAsyncDO	Continuous Digital Output is not set in asynchronous mode.
-29	ErrorNotInputPort	Value of AI/DI port argument is invalid.

Status Code	Name	
-30	ErrorNotOutputPort	Value of AO/DO argument is invalid.
-31	ErrorInvalidDioPort	Value of DI/O port argument is invalid.
-32	ErrorInvalidDioLine	Value of DI/O line argument is invalid.
-33	ErrorContIoActive	Continuous IO operation is not active.
-34	ErrorDbIBufModeNotAllowed	Double Buffer mode is not allowed
-35	ErrorConfigFailed	The specified function configuration failed.
-36	ErrorInvalidPortDirection	The value of DIO port direction argument is invalid.
-37	ErrorBeginThreadError	Failed to create thread.
-38	ErrorInvalidPortWidth	The port width setting is not allowed.
-39	ErrorInvalidCtrSource	The clock source setting is invalid.
-40	ErrorOpenFile	Failed to open file.
-41	ErrorAllocateMemory	Memory allocation failed.
-42	ErrorDaVoltageOutOfRange	Value of DA voltage argument is out of range.
-43	ErrorInvalidSyncMode	The sync. mode of operation is invalid.
-44	ErrorInvalidBufferID	The buffer id selected is invalid.
-45	ErrorInvalidCNTInterval	The counter value is invalid.
-46	ErrorReTrigModeNotAllowed	The re-trigger mode of operation is invalid.
-47	ErrorResetBufferNotAllowed	The buffer is not allowed to be reset.
-48	ErrorAnaTriggerLevel	The value of analog trigger level is invalid.
-49	ErrorDAQEvent	The DAQEvent is invalid.

Status Code	Name	
-50	ErrorInvalidDataSize	The data size is invalid. For example, the data size of pre-trigger and middle trigger must be a multiple of 32.
-51	ErrorOffsetCalibration	The AD offset calibration failed.
-52	ErrorGainCalibration	The AD gain calibration failed.
-53	ErrorCountOutOfSDRAMSize	The data count is out of size for the onboard SDRAM or DDR2.
-54	ErrorNotStartTriggerModule	The module is not installed in a start trigger slot.
-55	ErrorInvalidRouteLine	Value of routing line is invalid.
-56	ErrorInvalidSignalCode	Value of signal code is invalid.
-57	ErrorInvalidSignalDirection	The signal connection direction is invalid.
-58	ErrorTRGOSCalibration	The analog trigger level offset calibration failed.
-58	ErrorTRGOSCalibration	Analog Trigger offset calibration error
-59	ErrorNoSDRAM	No onboard SDRAM or DDR2
-60	ErrorIntegrationGain	The value of Integration Gain is invalid
-61	ErrorAcquisitionTiming	The value of acquisition time is invalid
-62	ErrorIntegrationTiming	The value of integration time is invalid
-63	ErrorInvalidTraceCnt	The value of trace count is invalid
-64	ErrorTriggerSource	The selected trigger source is invalid
-70	ErrorInvalidTimeBase	The selected TimeBase is invalid
-71	ErrorUndefinedParameter	One or more parameters are undefined
-80	ErrorNotDAQSteppedMode	The DAQ Stepped mode is not set
-90	ErrorBufAddrNotQuadDWordAlignment	The buffer address is not in QuadDWord Alignment

Status Code	Name	
-106	ErrorDevicelsRunning	Operation cannot be performed while device is running
-110	ErrorCalAddress	The calibration address is invalid
-111	ErrorInvalidCalBank	The selected calibration bank is invalid
-126	ErrorPLLNotLocked	PLL could not phase-lock to the reference clock
-127	ErrorCLKNotPresent	The selected clock source or timebase is not present
-201	ErrorConfigIoctl	The configuration API failed.
-202	ErrorAsyncSetIoctl	The async. mode API failed.
-203	ErrorDBSetIoctl	The double-buffer setting API failed.
-204	ErrorDBHalfReadyIoctl	The half-ready API failed.
-205	ErrorContOPIOctl	The continuous data acquisition API failed.
-206	ErrorContStatusIoctl	The continuous data acquisition status API setting failed.
-207	ErrorPIOIoctl	The polling data API failed.
-208	ErrorDIntSetIoctl	The dual interrupt setting API failed.
-209	ErrorWaitEvtIoctl	The wait event API failed.
-210	ErrorOpenEvtIoctl	The open event API failed.
-211	ErrorCOSIntSetIoctl	The COS interrupt setting API failed.
-212	ErrorMemMapIoctl	The memory mapping API failed.
-213	ErrorMemUMapSetIoctl	The memory unmapping API failed.
-214	ErrorCTRIoctl	The counter API failed.
-215	ErrorGetResIoctl	The resource getting API failed.

Table A-1: Status Codes

A.2 AI Range Codes

The following table lists the analog input range of digitizer cards.

Card	Input Range
AD_B_10_V	Bipolar -10V to +10V
AD_B_5_V	Bipolar -5V to +5V
AD_B_2_5_V	Bipolar -2.5V to +2.5V
AD_B_1_25_V	Bipolar -1.25V to +1.25V
AD_B_0_625_V	Bipolar -0.625V to +0.625V
AD_B_0_3125_V	Bipolar -0.3125V to +0.3125V
AD_B_0_5_V	Bipolar -0.5V to +0.5V
AD_B_0_05_V	Bipolar -0.05V to +0.05V
AD_B_0_005_V	Bipolar -0.005V to +0.005V
AD_B_1_V	Bipolar -1V to +1V
AD_B_0_1_V	Bipolar -0.1V to +0.1V
AD_B_0_01_V	Bipolar -0.01V to +0.01V
AD_B_0_001_V	Bipolar -0.01V to +0.001V
AD_U_20_V	Unipolar 0 to +20V
AD_U_10_V	Unipolar 0 to +10V
AD_U_5_V	Unipolar 0 to +5V
AD_U_2_5_V	Unipolar 0 to +2.5V
AD_U_1_25_V	Unipolar 0 to +1.25V
AD_U_1_V	Unipolar 0 to +1V
AD_U_0_1_V	Unipolar 0 to +0.1V
AD_U_0_01_V	Unipolar 0 to +0.01V
AD_U_0_001_V	Unipolar 0 to +0.001V
AD_B_2_V	Bipolar -2V to +2V
AD_B_0_25_V	Bipolar -0.25V to +0.25V
AD_B_0_2_V	Bipolar -0.2V to +0.2V
AD_U_4_V	Unipolar 0 to +4V
AD_U_2_V	Unipolar 0 to +2V
AD_U_0_5_V	Unipolar 0 to +0.5V
AD_U_0_4_V	Unipolar 0 to +0.4V
AD_B_1_5_V	Bipolar -1.5V to +1.5V

Card	Input Range
AD_B_0_2145_V	Bipolar -0.2145V to +0.2145V

Table A-2: Analog Digitizer Card Input Ranges

Valid values for each card:

Card	Values
PCI-9820/PXI-9820	AD_B_5_V, AD_B_1_V
PCI-9816D/PXI-9816D/PCle-9816D	AD_B_1_V, AD_B_0_2_V
PCI-9826D/PXI-9826D/PCle-9826D	
PCI-9846D/PXI-9846D/PCle-9846D	
PCI-9816H/PXI-9816H/PCle-9816H	AD_B_5_V, AD_B_1_V
PCI-9826H/PXI-9826H/PCle-9826H	
PCI-9846H/PXI-9846H/PCle-9846H	
PCI-9816V/PXI-9816V/PCle-9816V	AD_B_10_V, AD_B_2_V
PCI-9826V/PXI-9826V/PCle-9826V	
PCI-9846V/PXI-9846V/PCle-9846V	
PCle-9842	AD_B_1_V
PXIe-9848	AD_B_2_V, AD_B_0_2_V
PCle/PXIe-9814	AD_B_10_V, AD_B_5_V, AD_B_1_V, AD_B_0_5_V
PCle/PXIe-9852	AD_B_10_V, AD_B_2_V, AD_B_0_2_V

Table A-3: Valid Card Values

A.3 AI Data Format

This appendix lists the AI data format for the cards performing analog input operation, as well as the calculation methods to retrieve the A/D converted data and the channel where the data read from.

Card Type	Data Format	Value calculation
		- channel no. (CH#) - A/D converted data (ND) - Value returned from AI function (OD)
PCI-9820	Every 16-bit unsigned integer data: b15 b14 D13 D12 D11...D1 D0, where D13, D11,..., D0 is the A/D converted data, and b14 is the over-voltage indicator.	ND = OD & 0x3fff
PXI-9820	Every 16-bit unsigned integer data: b15 b14 D13 D12 D11...D1 D0, where D13, D11,..., D0 is the A/D converted data, and b15, b14 is the simultaneous digital input data.	ND = OD & 0x3fff
9816 9826 9846	Every 16-bit unsigned integer data: D15 D14 D13...D1 D0, where D15, D14,..., D0 is the A/D converted data,	ND = OD
9842	Every 16-bit signed integer data: D15 D14...D3 D2 SDI1 SDI0, where D15,..., D2 is the A/D converted data and SDI1, SDI0 is the SDI status if SDI mode is enabled	ND = OD>>2
9848	Every 16-bit signed integer data: D15 D14...D3 D2 b1 b0, where D15,..., D2 is the A/D converted data and b1, b0 are not applicable	ND = OD>>2
9852	Every 16-bit signed integer data: D15 D14...D3 D2 b1 b0, where D15,..., D2 is the A/D converted data and b1, b0 are not applicable	ND=OD>>2

Card Type	Data Format	Value calculation
		- channel no. (CH#) - A/D converted data (ND) - Value returned from AI function (OD)
9814	Every 16-bit signed integer data: D15 D14...D4 b3 SDI2 SDI1 SDI0, where D15,..., D4 is the A/D converted data, and SDI2, SDI1, SDI0 are the SDI status and b3 is not applicable	ND=OD>>4

A.4 Data File Format

This appendix describes the file format of the data files generated by the functions performing continuous data acquisition followed by storing the data to disk.

The data file includes three parts, Header, ChannelRange (optional), and Data block. The file structure is shown below:

Header
ChannelRange (Optional)
DAQ data

Header

The header part records the information related to the stored data and has 60 bytes of length. The data structure of the file header is listed in the table:

Header Total Length: 60 bytes			
Elements	Type	Size (bytes)	Comments
ID	char	10	File ID (ADLINKDAQ3)
card_type	short	2	Card Type
num_of_channel	short	2	Number of scanned channels
channel_no	unsigned char	1	Channel number where data is read from.
num_of_scan	long	4	Number of scan for each channel (total count / num_of_channel)

Header Total Length: 60 bytes			
Elements	Type	Size (bytes)	Comments
data_width	short	2	Data width (0: 8 bits, 1: 16 bits, 2: 32 bits)
channel_order	short	2	Channel scanned sequence 0: normal (0-1-2-3) 1: reverse (3-2-1-0) 2: custom* (0, 1, 3)
ad_range	short	2	AI range code. (0: AD_B_5V) Refer to Section A.2 AI Range Codes
scan_rate	double	8	Scanning rate of each channel (total sampling rate/num_of_channel)
num_of_channel_range	short	2	Number of ChannelRange* structure
start_date	char	8	Starting date of data acquisition (12/31/99)
start_time	char	8	Starting time of data acquisition (18:30:25)
start_millisecond	char	3	Starting millisecond of data acquisition (360)
reserved	char	6	Not used

- * If the num_of_channel_range is 0, the ChannelRange block will not be included in the data file.
- * The channel_order is set to "custom" only when the card supports variant channel scanning order.

Table A-4: Header Data Structure

Channel Range

The ChannelRange part records the channel number and data range information related to the stored data. This part consists of several channel and range units. The length of each unit is 2 bytes. The total length depends on the value of num_of_channel_range (one element of the file header) and is calculated with this formula:

Total Length = 2 * num_of_channel_range bytes

The data structure of each ChannelRange unit is as follows

ChannelRange (Unit Length: 2 bytes)			
Elements	Type	Size (bytes)	Comments
Channel	Char	1	Scanned channel number
Range	Char	1	AI range code of channel. (0: AD_B_5V) Refer to Section A.2 AI Range Codes

Table A-5: Channel Range

Data Block

The data is written to file in a 16-bit binary format, with the lower byte first (little endian). For example, the value 0x1234 is written to disk with 34 first followed by 12. The total length of the data block depends on the data width and the total data count.

The file is written in Binary format and may not be read by normal text editor. You can use any binary file editor to view it or the functions used for reading files (such as fread) to get the file information and data value. WD-DASK provides the DAQCvt utility to convert the binary file. Refer to the WD-DASK user guide for details.



Figure A-1: Data Conversion Utility

DAQCvt translates the information stored in the header part and the ChannelRange part, then displays the corresponding information in the **Input File** frame of DAQCvt main window. After setting the properties (File Path, Format, ...etc) of the converted file and after clicking the **Start Convert** button, DAQCvt gets rid of header and ChannelRange parts and converts the data in data block according to the card type and the data width. DAQCvt also writes the converted data to a disk and lets you use any text editor or Excel to view or analyze the accessed data.

Important Safety Instructions

For user safety, please read and follow all **instructions**, **WARNINGS**, **CAUTIONS**, and **NOTES** marked in this manual and on the associated equipment before handling/operating the equipment.

- ▶ Read these safety instructions carefully.
- ▶ Keep this user's manual for future reference.
- ▶ Read the specifications section of this manual for detailed information on the operating environment of this equipment.
- ▶ When installing/mounting or uninstalling/removing equipment:
 - ▷ Turn off power and unplug any power cords/cables.
- ▶ To avoid electrical shock and/or damage to equipment:
 - ▷ Keep equipment away from water or liquid sources;
 - ▷ Keep equipment away from high heat or high humidity;
 - ▷ Keep equipment properly ventilated (do not block or cover ventilation openings);
 - ▷ Make sure to use recommended voltage and power source settings;
 - ▷ Always install and operate equipment near an easily accessible electrical socket-outlet;
 - ▷ Secure the power cord (do not place any object on/over the power cord);
 - ▷ Only install/attach and operate equipment on stable surfaces and/or recommended mountings; and,
 - ▷ If the equipment will not be used for long periods of time, turn off and unplug the equipment from its power source.

- ▶ Never attempt to fix the equipment. Equipment should only be serviced by qualified personnel.

A Lithium-type battery may be provided for uninterrupted, backup or emergency power.



Risk of explosion if battery is replaced with one of an incorrect type. Dispose of used batteries appropriately.

-
- ▶ Equipment must be serviced by authorized technicians when:
 - ▷ The power cord or plug is damaged;
 - ▷ Liquid has penetrated the equipment;
 - ▷ It has been exposed to high humidity/moisture;
 - ▷ It is not functioning or does not function according to the user's manual;
 - ▷ It has been dropped and/or damaged; and/or,
 - ▷ It has an obvious sign of breakage.

Getting Service

Contact us should you require any service or assistance.

ADLINK Technology, Inc.

Address: 9F, No.166 Jian Yi Road, Zhonghe District
New Taipei City 235, Taiwan
新北市中和區建一路 166 號 9 樓
Tel: +886-2-8226-5877
Fax: +886-2-8226-5717
Email: service@adlinktech.com

Ampro ADLINK Technology, Inc.

Address: 5215 Hellyer Avenue, #110
San Jose, CA 95138, USA
Tel: +1-408-360-0200
Toll Free: +1-800-966-5200 (USA only)
Fax: +1-408-360-0222
Email: info@adlinktech.com

ADLINK Technology (China) Co., Ltd.

Address: 上海市浦东新区张江高科技园区芳春路 300 号 (201203)
300 Fang Chun Rd., Zhangjiang Hi-Tech Park
Pudong New Area, Shanghai, 201203 China
Tel: +86-21-5132-8988
Fax: +86-21-5132-3588
Email: market@adlinktech.com

ADLINK Technology Beijing

Address: 北京市海淀区上地东路 1 号盈创动力大厦 E 座 801 室(100085)
Rm. 801, Power Creative E, No. 1 Shang Di East Rd.
Beijing, 100085 China
Tel: +86-10-5885-8666
Fax: +86-10-5885-8626
Email: market@adlinktech.com

ADLINK Technology Shenzhen

Address: 深圳市南山区科技园南区高新南七道 数字技术园
A1 栋 2 楼 C 区 (518057)
2F, C Block, Bldg. A1, Cyber-Tech Zone, Gao Xin Ave. Sec. 7
High-Tech Industrial Park S., Shenzhen, 518054 China
Tel: +86-755-2643-4858
Fax: +86-755-2664-6353
Email: market@adlinktech.com

LiPPERT ADLINK Technology GmbH

Address: Hans-Thoma-Strasse 11, D-68163
Mannheim, Germany
Tel: +49-621-43214-0
Fax: +49-621 43214-30
Email: emea@adlinktech.com

ADLINK Technology, Inc. (French Liaison Office)

Address: 6 allée de Londres, Immeuble Ceylan
91940 Les Ulis, France
Tel: +33 (0) 1 60 12 35 66
Fax: +33 (0) 1 60 12 35 66
Email: france@adlinktech.com

ADLINK Technology Japan Corporation

Address: 〒101-0045 東京都千代田区神田鍛冶町 3-7-4
神田 374 ビル 4F
KANDA374 Bldg. 4F, 3-7-4 Kanda Kajicho,
Chiyoda-ku, Tokyo 101-0045, Japan
Tel: +81-3-4455-3722
Fax: +81-3-5209-6013
Email: japan@adlinktech.com

ADLINK Technology, Inc. (Korean Liaison Office)

Address: 137-881 서울시 서초구 서초대로 326, 802 (서초동, 모인터빌딩)
802, Mointer B/D, 326 Seocho-daero, Seocho-Gu,
Seoul 137-881, Korea
Tel: +82-2-2057-0565
Fax: +82-2-2057-0563
Email: korea@adlinktech.com

ADLINK Technology Singapore Pte. Ltd.

Address: 84 Genting Lane #07-02A, Cityneon Design Centre
Singapore 349584
Tel: +65-6844-2261
Fax: +65-6844-2263
Email: singapore@adlinktech.com

ADLINK Technology Singapore Pte. Ltd. (Indian Liaison Office)

Address: #50-56, First Floor, Spearhead Towers
Margosa Main Road (between 16th/17th Cross)
Malleswaram, Bangalore - 560 055, India
Tel: +91-80-65605817, +91-80-42246107
Fax: +91-80-23464606
Email: india@adlinktech.com

ADLINK Technology, Inc. (Israeli Liaison Office)

Address: 27 Maskit St., Corex Building
PO Box 12777
Herzliya 4673300, Israel
Tel: +972-54-632-5251
Fax: +972-77-208-0230
Email: israel@adlinktech.com

ADLINK Technology, Inc. (UK Liaison Office)

Tel: +44 774 010 59 65
Email: UK@adlinktech.com